

Universitatea „Sapientia” din Cluj-Napoca
FACULTATEA DE ȘTIINȚE TEHNICE ȘI UMANISTE,
TÎRGU MUREȘ
SPECIALIZAREA CALCULATOARE

Recunoașterea automată a instrumentelor muzicale pe baza înregistrărilor audio

PROIECT DE DIPLOMĂ

Coordonator științific:
Conf. univ. dr. Iclănzan David
Andrei

Absolvent:
Nagy Csaba

LUCRARE DE DIPLOMĂ

Coordonator științific:
Conf. univ. dr. Iclănzan David Andrei

Candidat: **Nagy Csaba**
Anul absolvirii: **2026**

a) Tema lucrării de licență:

Recunoașterea automată a instrumentelor muzicale pe baza înregistrărilor audio

b) Problemele principale tratate:

- Studiul teoretizat al procesării semnalelor audio și al algoritmilor de învățare profundă (Deep Learning).
- Construirea unui set de date hibrid prin fuziunea a șase baze de date publice (IRMAS, NSynth, Medley-solos-DB, TinySOL, Good-sounds, ESC-50) și înregistrări proprii realizate cu microfon.
- Extragerea și analiza caracteristicilor spectrale: Log-Mel spectrograme, Short-Time Fourier Transform (STFT) și Mel-Frequency Cepstral Coefficients (MFCC).
- Proiectarea, optimizarea și antrenarea unei arhitecturi de rețea neuronală convoluțională bidimensională (2D CNN) în mediul Google Colab.
- Dezvoltarea unui modul de augmentare a datelor (zgomot de fond, reverberație) pentru îmbunătățirea robusteții modelului în condiții acustice reale.
- Validarea sistemului prin implementarea unui modul de recunoaștere în timp real a sunetelor captate la microfon.

c) Desene obligatorii:

d) Softuri obligatorii:

- Prelucrarea setului de date audio în Python 3.
- Proiectarea și antrenarea rețelei neuronale 2D CNN cu TensorFlow și Keras în Google Colab.
- Extragerea caracteristicilor spectrale ale sunetelor utilizând biblioteca Librosa.
- Dezvoltarea modulului de recunoaștere în timp real a sunetelor cu SoundDevice.

e) Bibliografia recomandată:

f) Termene obligatorii de consultații: săptămânal

g) Locul și durata practicii: Universitatea Sapiientia,
Facultatea de Științe Tehnice și Umaniste din Târgu Mureș

Primit tema la data de:

Termen de predare:

Semnătura Director Departament

Semnătura coordonatorului

**Semnătura responsabilului
programului de studiu**

Semnătura candidatului



Declarație

Subsemnatul **Nagy Csaba**, absolvent al specializării **Calculatoare**, promoția **2026** cunoscând prevederile Legii Educației Naționale 1/2011 și a Codului de etică și deontologie profesională a Universității Sapientia cu privire la furt intelectual declar pe propria răspundere că prezenta lucrare de licență/proiect de diplomă/disertație se bazează pe activitatea personală, cercetarea/proiectarea este efectuată de mine, informațiile și datele preluate din literatura de specialitate sunt citate în mod corespunzător.

Târgu Mureș,

Data:

Semnătura:



Recunoașterea automată a instrumentelor muzicale pe baza înregistrărilor audio

Extras

Prezenta lucrare de diplomă abordează problema recunoașterii automate a instrumentelor muzicale pe baza înregistrărilor audio, utilizând metode moderne de învățare automată. Sistemul dezvoltat este capabil să clasifice în timp real șapte categorii de instrumente (chitară, pian, voce, instrumente cu coarde, instrumente cu ancie, instrumente de alamă și zgomot ambiental) pornind de la semnale audio monofonice.

Abordarea propusă se bazează pe extragerea a trei tipuri de caracteristici spectrale (Log-Mel spectrograme, STFT și MFCC) și pe antrenarea unor rețele neuronale convoluționale bidimensionale (2D CNN) în mediul Google Colab. Setul de date hibrid a fost creat prin combinarea unor eșantioane curate descărcate de pe internet, a unor re-înregistrări proprii prin microfon și a zgomotului ambiental din baza de date ESC-50, iar un modul de augmentare a datelor simulează condițiile reale de captare prin adăugarea de reverberație, zgomot ambiental și distorsiuni de microfon.

Rezultatele experimentale arată o acuratețe competitivă a modelului pe datele de test, iar sistemul a fost validat prin recunoașterea în timp real a sunetelor captate cu microfonul laptopului. Lucrarea contribuie la demonstrarea fezabilității unui clasificator robust de instrumente muzicale, antrenat cu resurse computaționale accesibile.

Cuvinte cheie: recunoașterea instrumentelor muzicale, rețele neuronale convoluționale, Mel-spectrogramă, STFT, MFCC, clasificare audio, formă de undă, imagine spectrală

Sapientia Erdélyi Magyar Tudományegyetem

MAROSVÁSÁRHELYI KAR

SZÁMÍTÁSTECHNIKA SZAK

Automatikus hangszerfelismerés zenefelvételek alapján

Diplomadolgozat

Témavezető:

**Dr. Iclănzan David Andrei,
egyetemi docens**

Végzős hallgató:

Nagy Csaba

2026

Automatikus hangszerfelismerés zenefelvételek alapján

Kivonat

Jelen diplomadolgozat az automatikus hangszerfelismerés problémáját vizsgálja zenei felvételek alapján, modern gépi tanulási módszerek alkalmazásával. A kidolgozott rendszer valós időben képes osztályozni hét hangszerkategóriát (gitár, zongora, ének, vonós, nádas fúvós, rézfúvós és környezeti zaj) monofón audiojel alapján.

A megvalósított megoldás három spektrális jellemző típust (Log-Mel spektrogram, STFT és MFCC) használ, amelyek kétdimenziós konvolúciós neurális hálózatba (2D CNN) kerülnek betáplálásra. A modell tanítása Google Colab környezetben történt. Az adathalmazt saját tiszta internetes minták és mikrofonnal/telefonnal újrafelvételezett minták, valamint az ESC-50 környezeti zajadatbázis egyesítésével hoztam létre. Az adataugmentációs modul a valós felvételi körülményeket szimulálja visszhang, környezeti zaj és mikrofonkarakterisztika hozzáadásával.

A kísérleti eredmények versenyképes osztályozási pontosságot mutatnak a tesztadatokon, és a rendszert valós idejű mikrofonos felismeréssel is validáltam. A dolgozat hozzájárul annak demonstrálásához, hogy elérhető számítási erőforrásokkal is megvalósítható egy robusztus hangszerosztályozó rendszer.

Kulcsszavak: hangszerfelismerés, konvolúciós neurális hálózat, Mel-spektrogram, STFT, MFCC, audioosztályozás, hullámforma, spektrális kép

Automatic Musical Instrument Recognition Based on Audio Recordings

Abstract

This thesis addresses the problem of automatic musical instrument recognition from audio recordings using modern machine learning methods. The developed system is capable of classifying seven instrument categories in real time (guitar, piano, vocals, strings, reeds, brass, and ambient noise) from monophonic audio signals.

The proposed approach relies on the extraction of three types of spectral features (Log-Mel spectrograms, STFT, and MFCC), which are fed into two-dimensional convolutional neural networks (2D CNN) trained in the Google Colab environment. A hybrid dataset was assembled by combining clean internet-sourced samples, self-recorded microphone playback samples, and ambient noise from the ESC-50 database. A data augmentation module simulates real-world recording conditions by adding reverberation, ambient noise, and microphone characteristics.

Experimental results demonstrate competitive classification accuracy on the test set, and the system has been validated through real-time recognition of sounds captured by a laptop microphone. This work contributes to demonstrating the feasibility of building a robust musical instrument classifier using accessible computational resources.

Keywords: musical instrument recognition, convolutional neural network, Mel-spectrogram, STFT, MFCC, audio classification, waveform, spectral image

Előszó

Amióta az eszemet tudom, a reáltudományok és a művészetek – különösen a zene – világa meghatározó szerepet játszik az életemben. Időről időre az egyik kerül előtérbe, majd a másik veszi át a szerepet, de sosem maradt ki teljesen egyik sem az érdeklődésemből. Ez az oka, hogy sosem tudtam csak egy oldal mellett elköteleződni véglegesen és minden fókuszomat kizárólag ráhelyezni, hogy minél jobb legyek benne. Talán nincs is rá szükség. A reáltudományok segítenek leírni a körülöttem lévő fizikai valóságot és lépést tarthatok vele a technológia exponenciális előretörésével. Ennek ellenére mégis hiányérzetem támad, ha a saját belső világom szeretném megfigyelni. A művészetek körülírják és teljesebbé teszik az ember lelkivilágát, így teremtve meg számomra az egyensúlyt.

Ez az egyensúlykeresés inspirálta jelen kutatásomat és diplomadolgozatomat is, amely a zene és a számítástechnika határterületén valósul meg.

Nagy Csaba

TARTALOMJEGYZÉK

Előszó	iii
Tartalomjegyzék	iv
Ábrák jegyzéke	vi
Táblázatok jegyzéke	vii
1. Bevezető	1
2. Elméleti megalapozás és szakirodalmi tanulmány	3
2.1. Szakirodalmi tanulmány	3
2.2. Elméleti alapok	3
2.3. Ismert hasonló alkalmazások	3
2.4. Felhasznált technológiák	3
3. A rendszer specifikációi és architektúrája	4
4. Részletes tervezés	5
4.1. Az adathalmaz tervezése	5
4.1.1. Saját készítésű organikus adathalmaz	5
4.1.2. Mikrofonos újrafelvétel a robusztusság érdekében	7
4.1.3. Végző adathalmaz összeállítása	9
4.2. Jellemzőkinyerés tervezése	11
4.2.1. Log-Mel spektrogram	11
4.2.2. STFT spektrogram	12
4.2.3. MFCC (Mel-Frequency Cepstral Coefficients)	12
4.2.4. A megfelelő jellemzőkinyerési eljárás kiválasztása	12
4.2.5. Normalizálás	13
4.2.6. Jellemzőkinyerés kimeneti állományai	14
4.3. Adataugmentáció tervezése	14
4.3.1. Szobai reverb szimuláció	14
4.3.2. Frekvenciaválasz-torzítás (Pre-emphasis)	14
4.3.3. Környezeti zajkeverés (Noise mixing)	15
4.3.4. Szoftklip torzítás (Soft clipping)	15
4.3.5. Végző normalizálás	15
4.4. A 2D CNN hálózat architektúrájának tervezése	17
4.4.1. A hálózati architektúra részletei	17
4.4.2. Tanítási konfiguráció	18
4.4.2.1. A tanítás során generált kimenetek	19

4.5.	A valós idejű felismerő modul tervezése	20
4.5.1.	Audio stream és aszinkron pufferek	20
4.5.2.	Jellemzőkinyerés és normalizálás	20
4.5.3.	Zajkapu és csenddetektálás	21
4.5.4.	Predikciós döntési logika és hiszterézis	21
4.6.	A feldolgozási folyamat összefoglaló áttekintése	23
5.	Üzembe helyezés és kísérleti eredmények	25
5.1.	Üzembe helyezési lépések	25
5.2.	Felmerült problémák és megoldások	26
5.3.	Kísérleti eredmények és mérések	26
6.	A rendszer felhasználása	28
6.1.	Az alkalmazás indítása és konfigurációja	28
6.2.	A parancssori felhasználói felület felépítése	28
6.2.1.	ANSI színkódolás és jelölések	29
6.3.	Működési forgatókönyvek és tesztesetek	29
6.3.1.	1. teszteset: Csendes szobai környezet (alapállapot)	29
6.3.2.	2. teszteset: Akusztikus gitárjáték (monofón dallam)	30
6.3.3.	3. teszteset: Emberi énekhang (vokál)	30
6.3.4.	4. teszteset: Szaxofon játék (átmeneti bizonytalanság)	30
6.3.5.	Az alkalmazás leállítása	30
7.	Következtetések	32
7.1.	Megvalósítások	32
7.2.	Hasonló rendszerekkel való összehasonlítás	32
7.3.	Továbbfejlesztési lehetőségek	33
8.	Irodalomjegyzék	34
9.	Függelékek	35
9.1.	A projekt könyvtárszerkezete	35
9.2.	A feldolgozási csővezeték szkriptjei	36
9.3.	Fejlesztői és futtatási környezet	37

ÁBRÁK JEGYZÉKE

4.1.1.A saját gyűjtésű organikus adathalmaz osztályonkénti eloszlása a szeletelés után	7
4.1.2.A mikrofonos újrafelvételi és szinkronizált szeletelési folyamat lépései . . .	8
4.1.3.A saját tiszta és az újrafelvételezett zajos minták megoszlása az adathalmazban	9
4.1.4.A saját alapadatokból és adataugmentációból építkező hibrid adathalmaz folyamatdiagramja	10
4.1.5.A végső, kiegyensúlyozott hibrid adathalmaz belső összetétele kategóriák szerint	11
4.2.1.A három vizsgált jellemzőtípus vizualizációja ugyanazon gitárminta esetén: (a) STFT spektrogram (513×32), (b) normalizált Log-Mel spektrogram (64×32), (c) normalizált MFCC együtthatók (40×32)	13
4.3.1.A MacBook/mikrofonos felvételi körülményeket szimuláló adataugmentációs folyamatlánc	15
4.3.2.A hat hangszerosztály (gitár, zongora, vokál, hegedű, klarinét, kürt) egy- egy reprezentatív mintájának normalizált Log-Mel spektrogramja az eredeti (tiszta) és az adataugmentált (MacBook/mikrofon szimulációs láncsal torzított) állapotban	16
4.4.1.A tervezett 2D CNN hálózati architektúra felépítése és adatfolyama	18
4.5.1.A valós idejű felismerési folyamat lépései az élő hangbeviteltől a kijelzésig .	22
4.6.1.A hangszerfelismerő rendszer teljes feldolgozási folyamatának lépései . . .	24
5.3.1.A Log-Mel spektrogram alapú 2D CNN modell konfúziós mátrixának hőképe	26
6.1.1.Az alkalmazás indításakor megjelenő konfigurációs és betöltési paraméterek	28
6.3.1.A rendszer kimenete csendes szobai környezet (zaj) esetén	30
6.3.2.A rendszer kimenete akusztikus gitárjáték érzékelésekor	30
6.3.3.A rendszer kimenete emberi énekhang (vokál) érzékelésekor	30
6.3.4.A rendszer kimenete szaxofon játék (reed) érzékelésekor	30

TÁBLÁZATOK JEGYZÉKE

4.1.1.A vizsgált publikus adathalmazok összehasonlító elemzése	6
4.2.1.A három vizsgált jellemzőkinyerési módszer összehasonlító elemzése . . .	13
4.4.1.A tervezett 2D CNN architektúra rétegei (Log-Mel bemenet: $64 \times 32 \times 1$) . .	17
4.4.2.A modell tanításához használt konfigurációs paraméterek és beállítások . .	19
6.2.1.A parancssori felületen alkalmazott ANSI színkódok és rövidítések kategóri- ánként	29
9.1.1.A projekt legfelső szintű könyvtárszerkezete	35
9.2.1.A feldolgozási csővezeték szkriptjei és feladataik	36
9.3.1.A fejlesztői környezet főbb összetevői	37

1. BEVEZETŐ

Kodály Zoltán szerint a zene hangzó matematika. Hasonló analógiával élve, a számítástechnika pedig a digitális matematika egy formája, ami számomra azt mutatja meg, hogy valójában mindkét oldal közös tőről fakad.

Kutatásom és ezen dolgozat elkészítése pontosan ebben a metszéspontban, a zene és a számítástechnika határterületén valósul meg. Ezt a közös határterületet a tudományos világban a zenei információ-visszanyerés (Music Information Retrieval – MIR) interdiszciplináris területe képviseli, amely a digitális jelfeldolgozás és a gépi tanulás eszköztárát ötvözi, hogy képessé tegye a számítógépes rendszereket az audiojelek automatikus értelmezésére. Ezen belül az automatikus hangszer- és hangszercsalád-felismerés az egyik legizgalmasabb kutatási irány, amely kulcsfontosságú a digitális zenei adatbázisok intelligens kereshetőségéhez, a valós idejű zeneelemzéshez, valamint az automatikus kottázáshoz (transzkripcióhoz).

Hétköznapi szinten ennek a technológiának az egyik leglátványosabb alkalmazása a modern zenei streaming szolgáltatók (mint például a Spotify vagy az Apple Music) ajánló-rendszereiben érhető tetten. Ezek a platformok kifinomult algoritmusok segítségével elemzik és szabják személyre az általunk preferált zenei mintázatokat, hangszereket és dalokat, megteremtve ezzel a zenehallgatás új, intelligens korszakát. Ennek a személyre szabási folyamatnak a legalapvetőbb lépcsőfokát az olyan alacsony szintű zenei attribútumok és akusztikai jellemzők automatikus felismerése alkotja, mint amilyen a felvételeken megszólaló hangszerek azonosítása is.

A zenében a hangszerek változatos környezetben szólhatnak meg. Háttér szempontjából beszélhetünk „laboratóriumi” stúdiófelvételekről vagy valós idejű, környezeti háttérzajjal terhelt jelekről. A stúdióminőséghez jó analógia a légüres tér, ahol ideális körülmények között csak igencsak minimális zaj található meg, míg normális esetben szinte mindig zaj kíséri a zenét – ha más nem, legalább olyan természeti hangok, mint a szél, a madárcsicsergés vagy a lépések zaja, illetve magának a hangszernek a fizikai megszólaltatása, ami a hangfeldolgozás esetében a második legnagyobb probléma. A hangszerek megszólalhatnak egyedül, de akár többen is egyszerre: több hangszer együttes megszólalása egymásra adódik, úgymond összeolvad, aminek a szétválasztása és megkülönböztetése – mint a hangszerfelismerés „első számú károkozója” – különösen orkesztrák esetén szinte lehetetlen.

A probléma orvoslására az utóbbi időkben a deep learning modellek, különösen a 2D CNN hozott nagy könnyebbiséget, amely a hang szeleteiből hoz létre egy-egy spektrális képet, mintegy a hangok ujjlenyomataként. A hangokat képpé alakítva, a vizuális mintázatok és karakterisztikák alapján történő elemzés lehetővé teszi a hangszerek sajátos színezetének felismerését, ám a szélsőségesen komplex zenekaroknál ez a módszer sem jelent teljes áttörést.

A kutatáshoz használt adathalmazt saját magam állítottam össze az interneten fellelhető zenefelvételek egészen apró szeleteiből, miután alaposan áttekintettem a korábban létező adatbázisokat. Azok a meglévő minták ugyanis nem követték pontosan az én tervezési szándékaimat, és nem adtak választ azokra a specifikus problémákra, amelyeket vizsgálni

szerettem volna. Hosszú távon sokkal célravezetőbb egy teljesen saját, organikus adatbázis felépítése, amely tökéletesen illeszkedik a rendszer egyedi igényeihez.

Az elkészült neurális hálózatra és a saját adathalmazra építve a kutatás végcélja egy olyan alkalmazás kifejlesztése, amely közvetlen, gyakorlati segítséget nyújthat a felhasználóknak. Egy ilyen rendszer továbbfejlesztett változata különösen hasznos lehet azok számára, akik ritkán fordultak meg valós, élő zenei környezetben, és fejletlen a hangszerfelismerési képességük, de gyakran találják magukat olyan helyzetben, amikor egy számukra tetsző, videóklip nélküli dallamban megszólaló hangszer nevét szeretnék azonosítani.

A dolgozat további fejezetei lépcsőről lépésre követik a szoftverfejlesztés lépéseit. A zenei és gépi tanulási elmélet megalapozása után részletesen bemutatásra kerül a rendszer specifikációja és architektúrája, amelyet a 2D CNN hálózat és a valós idejű alkalmazás implementációja, valamint a gyakorlati tesztelés és kiértékelés követ.

2. ELMÉLETI MEGALAPOZÁS ÉS SZAKIRODALMI TANULMÁNY

Ez a fejezet tartalmazza az elvégzett dokumentálódás, bibliográfiai munka eredményét: a téma elméleti hátterét, hasonló alkalmazások és megoldások rövid ismertetőjét, amelyek már léteznek. A fejezetet alfejezetekre lehet bontani, **megfelelő címekkel** a témától függően.

2.1. Szakirodalmi tanulmány

2.2. Elméleti alapok

2.3. Ismert hasonló alkalmazások

2.4. Felhasznált technológiák

Ide kerül azoknak a technológiáknak, elméleti ismereteknek a rövid bemutatása, ami szükséges a megvalósítás megértéséhez. **Figyelem!** Ez a fejezet egy szintézis kell legyen, nem kimásolt szövegek. Mindennek az eredetét bibliográfiai hivatkozással kell jelezni [1], illetve a szó szerint átvett részeket idézőjelbe kell tenni a forrás megjelölésével. **Terjedelem: 5–12 oldal.**

3. A RENDSZER SPECIFIKÁCIÓI ÉS ARCHITEKTÚRÁJA

Itt be kell mutatni a rendszer specifikációját, valamint blokkséma szinten az architektúráját. **Terjedelem: 10–15 oldal.**

4. RÉSZLETES TERVEZÉS

A következő fejezetben részletes betekintés nyújtok, organikus adathalmazzal rendelkező, saját célú hangszerfelismerő rendszer felépítésébe, ismertetve minden tervezési lépést. Ez többek között tartalmazza az adatkészlet létrehozását, a jellemzési paramétereket, az adatbővítési beállításokat és a 2D konvolúciós neurális hálózatokat, végül a valós idejű hangszerfelismerő modul tervezését.

4.1. Az adathalmaz tervezése

Az adathalmaz szigorú szempontrendszer szerint került összeállításra, különös figyelmet szentelve a változatos és felismerhető hangszerjátéokra.

Az osztályrendszert hét osztályra osztottuk, amelyek a könnyű és klasszikus zenében előforduló leggyakoribb hangszertípusokat fogják reprezentálni. Van egy extra, környezeti zaj osztály is, amely segítségével kezeljük a csendet és a váratlan hangzavart. A 7 osztály a következő:

- (1) Gitár: tartalmazza az akusztikus és a kevésbé torzított elektromos gitárokat is
- (2) Zongora: tartalmazza a klasszikus és digitális zongorák egyes típusait
- (3) Vokál: tartalmaz emberi éneklést, rapet és egyéb sajátos hangképzéseket (mongol torokének, szájdob)
- (4) Húr: a vonós négyes hangszerei: hegedű, brácsa, cselló, bőgő
- (5) Nádfúvósok: mind egynádas (klarinét, szaxofon), mind kétnádas (fagott, oboa)
- (6) Rézfúvósok: legelterjedtebb változatok: trombita, harsona, kürt, tuba
- (7) Zaj: Háttér- és/vagy környezeti zajok (szél, eső, forgalom stb.), madárcsicsergés stb.

4.1.1. Saját készítésű organikus adathalmaz

Kezdetben a létező adathalmazokat vizsgáltam, hogy megértsem, hogyan érdemes elkezdeni a tervezési folyamatot, és megfigyeljem, miben erősek külön-külön. Ennek érdekében az előkészítő szakaszban lefuttattam egy elemző szkriptet (`01_analyze_datasets.py`), amely feltárta az elérhető források struktúráját. Az elemzés eredményeit a 4.1.1. táblázat foglalja össze.

4.1.1. táblázat: A vizsgált publikus adathalmazok összehasonlító elemzése

Adathalmaz	Tartalom leírása	Fő erősségek	Korlátok / Miért nem felelt meg
IRMAS [2]	Zenei részletek (3 s) szóló hangszeres és polifón szekciókkal	Valós zenei környezet, polifónia vizsgálata	Korlátozott számú és egyenetlen eloszlású hangszerkategóriák.
NSynth [3]	300 000+ db egyedi hangszeres hangjegy (4 s)	Hatalmas mintaállomány, egységes hanghossz és pitch	Csak izolált hangok, nincs benne valós zenei folytonosság és frazeálás.
Medley-solos-DB [4]	Különböző sávokból ki-nyert szóló hangszeres részek (3 s)	Valós zenei előadásokból származó, tiszta szóló felvételek	Alacsony mintaszám egyes kategóriákban, szűkös osztályválaszték.
TinySOL [5]	Izolált zenekari hangszerek hangjai, különböző technikákkal	Kiváló stúdióakusztika, gazdag játéktechnikai annotációk	Kizárólag izolált hangjegyek, nem reprezentálja a folyamatos zenélést.
Good-sounds [6]	Fúvós és vonós hangszerek egyedi hangjegyei	Különböző minőségű mikrofonok és felvételi minőségek	Nagyon szűk hangszerpaletta (pl. nincs gitár vagy énekhang).
ESC-50 [1]	2000 db környezeti és háttérzaj minta (5 s)	50 különböző zajkategória rendkívül széles spektruma	Nem tartalmaz zenei hangszereket, csak a zaj kategóriához alkalmas.

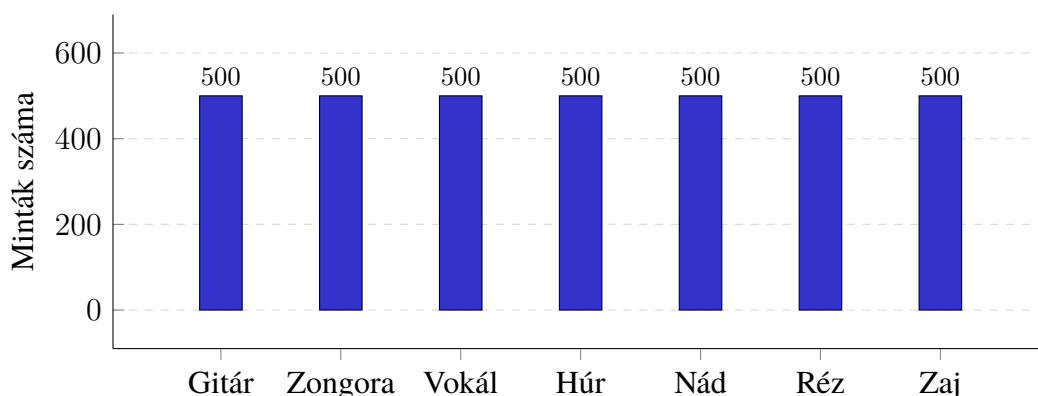
Megállapítottam, hogy a meglévő adathalmazok teljes vagy részleges felhasználása nem felel meg az igényeimnek. Minden adathalmaz mintái eltérő hosszúságúak, más a hangszerosztályok eloszlása, rengeteg mintát tartalmaznak, és különböző céllal jöttek létre. Emiatt – bár kiváló tervezési alapul szolgáltak – a feldolgozásuk egységesítése komplex feladat lenne, és az átválogatásuk is rengeteg időt igényelne. Egy organikus, a saját elképzeléseim alapján összeállított adathalmaz biztosabb és átláthatóbb alapokat nyújt egy jól működő modell megteremtéséhez. Ez alól kivételt képezett az ESC-50, amely sokféle környezeti és háttérzaj-mintát foglal magába, így hiánypótlóként szolgált egy mindenre kiterjedő zajos adathalmaz összeállításához.

Az interneten található zenei felvételekből válogattam, figyelmet fordítva az aktuális hangszerek karakterességének kiemelésére és a zenei játék változatosságára. Minden hangszerosztályhoz (gitár, zongora, ének, vonós, nád- és rézfúvós) nagyjából 100–150 darab interneten elérhető felvételt választottam ki. Minden feldolgozott felvételből a zenei tartal-

lom függvényében egy vagy két 5 másodperces részletet vágtam ki, amelyet egy előkészítő szkript (`owndataset/dataset.py`) szeletelt fel 1 másodperces, átfedésmentes mintákra. Ez osztályonként 500 darab, többnyire monofón, stúdió- és szobaminőségű mintát eredményezett.

Fontosnak tartom hangsúlyozni, hogy az internetről gyűjtött zenei hangmintákat kizárólag kutatási és oktatási (akadémiai) célokra használok fel. Ezeket a mintákat önmagukban nem teszem közzé, nem terjesztem nyilvánosan, és semmilyen kereskedelmi vagy pénzszerzési célra sem használok fel. Ez a megközelítés teljesen analóg a nagy technológiai vállalatok által alkalmazott gyakorlattal, ahol a nyilvánosan elérhető adatokat szigorúan a modellek tanítására használják fel (a szabad felhasználás, azaz a *fair use* elve alapján), miközben maguk a nyers forrásfájlok zártak maradnak.

Az adathalmaz eloszlását ezen lépés után a 4.1.1. ábra szemlélteti.



4.1.1. ábra: A saját gyűjtésű organikus adathalmaz osztályonkénti eloszlása a szeletelés után

4.1.2. Mikrofonos újrafelvétel a robusztusság érdekében

A kezdeti adatkészleteim önmagukban kevés zajos mintát tartalmaznak, melyeket az eredeti felvételt készítő eszköz mikrofonjával rögzítettek. Minden mikrofonnak saját jellemzői vannak: frekvencia-válasza, érzékenysége, torzítása, belső zaja és maximális hangnyomásszintje (SPL). Mivel a projektem célja a valós idejű hangszerfelismerés is, a modell stabilitásának kialakításához elengedhetetlen az optimalizálás a saját eszközünkre, amellyel a predikció megtörténik.

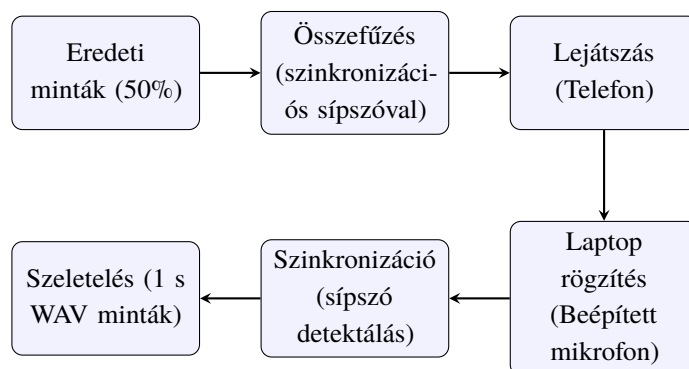
A robusztusság biztosítására egy speciális eljárást találtam ki. Mivel nem férek hozzá, és játszani sem tudok ennyiféle hangszereken, a telefon hangszóróján játszom vissza a mintákat a laptopom beépített mikrofonjába. A feljátszás 10–30 cm távolságról történik a beépített mikrofon pontos helyzetétől mérve, hogy a hang a közelség miatt se legyen túl torz, de a távolság miatt se váljon értelmezhetetlenné vagy túl halkká.

Ismételten kiválogatni és a telefonon egyenként feljátszani osztályonként 500 mintát, majd így kialakítani az optimális zajos adathalmazt, költséges és meg nem térülő folyamat. Ennek kiküszöbölésére azt találtam ki, hogy egy automatizációs script (`02_mic_prep_concatenate.py`) segítségével osztályonként összefűzöm a minták 50%-át (ez az arány biztosítja a megfelelő egyensúlyt a tiszta stúdióminőségű és a mikrofonnal rögzített zajos minták között, megakadályozva a laptop saját torzításaira való túltanulást), majd ezt telefon-

ról visszajátszva felvételezem (03_mic_record_audio.py), és szeletelem (04_mic_slice_audio.py) hasonlóan az internetes adathalmaz kialakításánál.

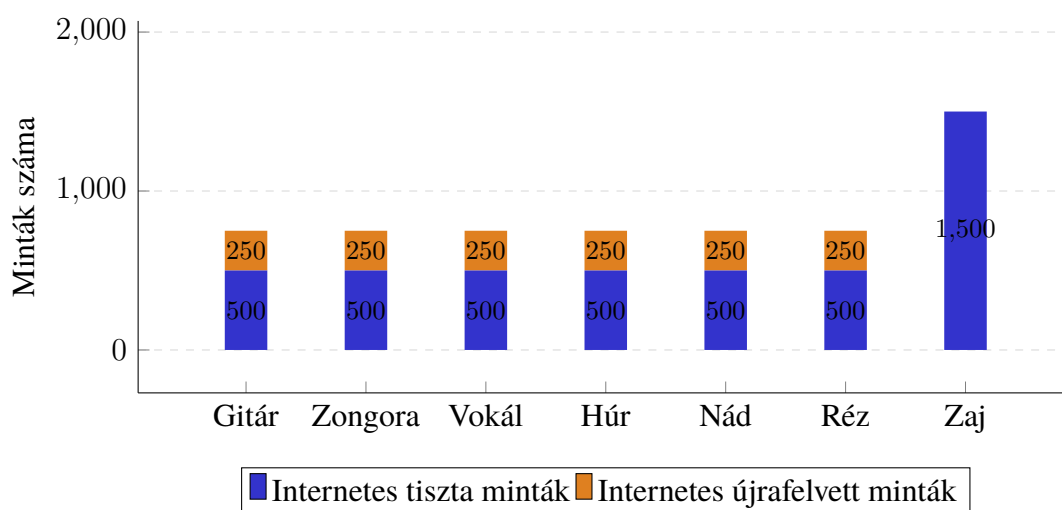
Ebben az esetben azonban a telefon lejátszásának és a laptop beépített mikrofonjának felvételezésének szinkronizációjára van szükség. A pontos szinkronizáció megvalósításához a következő folyamatot alakítottam ki: a 02_mic_prep_concatenate.py scripttel az összefűzött hanganyag legelejére egy 1 másodperces, 440 Hz-es szinuszos szinkronizációs sípszót generálok, a hangszeres mintákat pedig 0,2 másodperces csendes szakaszokkal választom el egymástól. A telefonról történő visszajátszás alatt a laptopomon futó 03_mic_record_audio.py script rögzíti a beérkező jelet 16 000 Hz-es mintavételezéssel, mono formátumban. Végül a 04_mic_slice_audio.py script a Librosa könyvtár segítségével detektálja a felvétel elején található sípszó végét, majd ettől a ponttól kiindulva szeleteli fel a hanganyagot pontosan 1 másodperces egységekre, miközben automatikusan kiszűri a csendes részeket.

Az automatizált mikrofonos újrafelvételi és szeletelési munkafolyamatot a 4.1.2. ábra szemlélteti.



4.1.2. ábra: A mikrofonos újrafelvételi és szinkronizált szeletelési folyamat lépései

Az újrafelvételi folyamat eredményeképpen az eredeti, tiszta minták mellé osztályonként további 250 darab (összesen 1500 darab) mikrofonos karakterisztikájú minta kerül. Ez a bővítmény hordozza a laptopom beépített mikrofonjának torzításait, frekvencia-választát és belső zaját, biztosítva ezzel a modell későbbi robusztusságát. Az adathalmaz aktuális összetételét és a minták felhalmozódását a 4.1.3. ábra szemlélteti egymásra épülő (stacked) formában, különválasztva az internetes tiszta és az internetes újrafelvett minták arányát. Fontos megjegyezni, hogy a Zaj osztály esetében nem végeztem ilyen újrafelvételt: az ebből a kategóriából származó ESC-50 minták eleve valós környezeti és háttérzajokat tartalmaznak, így teljesen feleslegesnek tartottam a szélzaj vagy az eső hangját telefonról visszajátszva újra felvenni, ráadásul ez a lejátszási lánc miatt szükségtelen egyedi torzulásokat is vinne a tiszta zajmintákba.



4.1.3. ábra: A saját tiszta és az újrafelvételezett zajos minták megoszlása az adathalmazban

4.1.3. Végső adathalmaz összeállítása

Egy konzisztens és pontos adathalmaz nagyban segíti a neurális hálózat tanulási folyamatát, ami jelentősen növeli az osztályozási pontosságot. Abban az esetben ugyanis, ha egy hangszerosztály lényegesen több mintával reprezentált, mint a többi, a hálózat hajlamos arra a hangszerkategóriára torzítani, amelyet gyakrabban látott a tanítás során. Ezért tartottam fontosnak biztosítani az osztályok egyenlő reprezentációját azáltal, hogy az egyes hangszerosztályok mintaszámát egységesen 1500 darab 1 másodperces mintára állítottam be. Tekintettel arra, hogy 7 hangszerosztályt határoztam meg, ez összesen 10 500 mintát jelent a teljes hibrid adathalmazban.

A hat hangszerosztály (gitár, zongora, ének, vonós, nád- és rézfúvós hangszerek) esetében nem használtam fel meglévő, nyilvános zenei adatbázisokat (mint pl. az IRMAS vagy az NSynth), hanem a teljes állományt magam állítottam össze. A kiinduló alapadathalmazt osztályonként 750 darab minta alkotja, amelyet két forrásból válogattam össze:

1. **Saját tiszta minták (500 db/osztály):** Különböző internetes forrásokból letöltött stúdió- és jó minőségű otthoni felvételek, amelyeket manuálisan vágtam ki és osztottam fel 1 másodperces szakaszokra.
2. **Mikrofonos újrafelvételek (250 db/osztály):** A saját tiszta mintáim 50%-ának (250 darab mintának) a laptop beépített mikrofonjával vagy mobiltelefonnal történő visszajátvása és rögzítése a valós környezet akusztikai torzításainak és zajainak modellezésére.

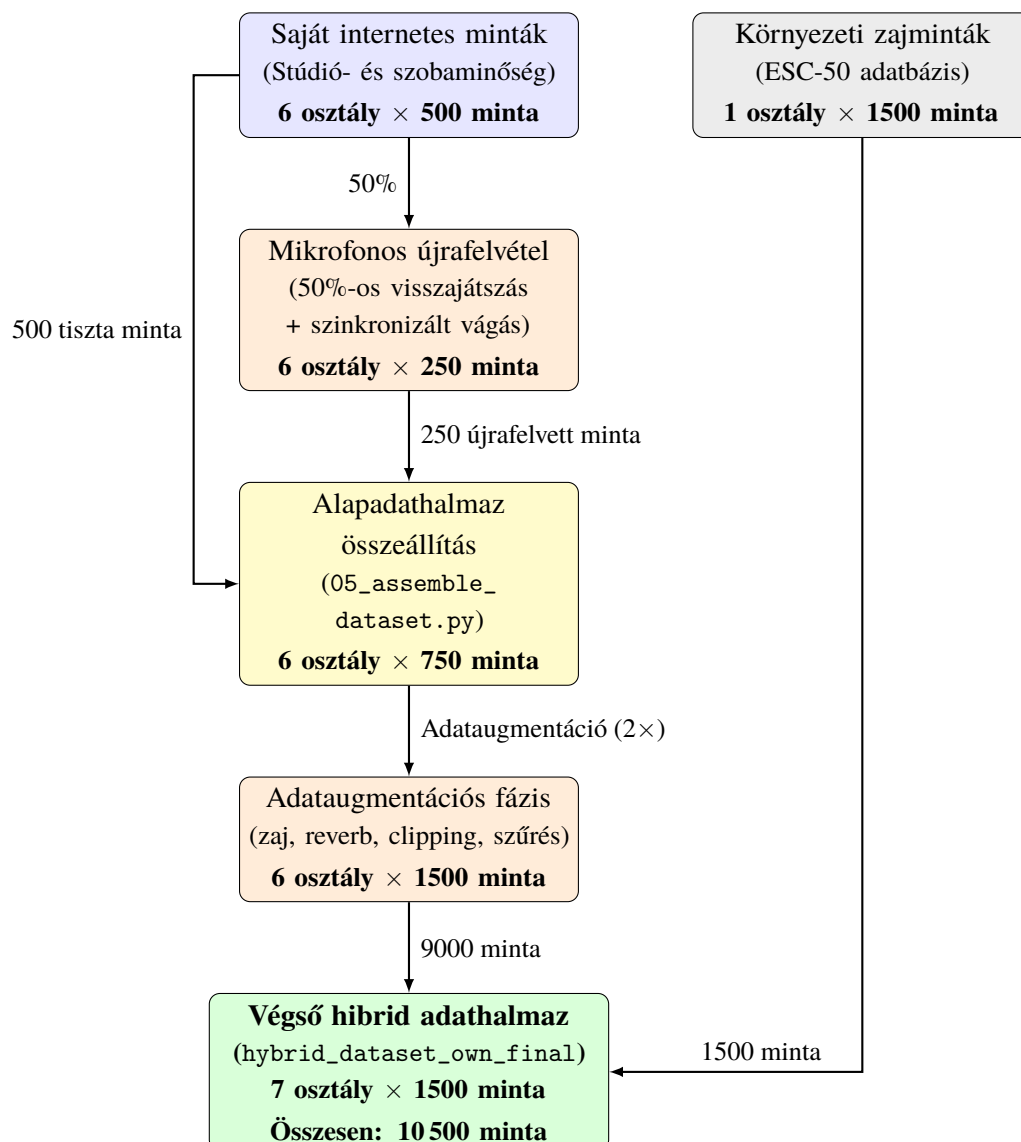
Az így kapott 750 alaplanta darabszámát adataugmentációs eljárásokkal dupláztam meg, létrehozva további 750 augmentált mintát osztályonként. Az augmentációs lánc során szobai zengetés szimulációt, pre-emphasis szűrést, környezeti zajkeverést és szoftklip torzítást alkalmazok – ezek pontos paramétereit és képleteit a 4.3. fejezet fejt ki részletesen. Így értem el a kívánt 1500 darabos mintaszámot a hangszerosztályoknál.

A noise (zaj) osztály esetében nem volt szükség ilyen jellegű megosztásra és adataugmentációra: ide közvetlenül az ESC-50 környezeti zajadatbázisból válogattam be 1500 kü-

lönböző mintát (például szél, eső, utcai zajok, egyéb háttérzajok), mivel ezek a felvételek a természetükből adódóan már eleve tartalmazznak mikrofon- és háttérzajokat.

A minták fizikai rendezését és könyvtárstruktúrába szervezését a `hybrid_dataset_own_final` mappába az `05_assemble_dataset.py` szkript segítségével végeztem el. A szkript a másolás megkezdése előtt véletlenszerűen összekeveri a forráskönyvtárakban található WAV fájlok listáját (`random.shuffle`), majd a kívánt darabszám eléréséig másolja a fájlokat az új, strukturált könyvtárakba. Ezzel garantálom a torzításmentes eloszlást. Minden fájlt egységesen mono csatornás, 16 000 Hz-es mintavételezési frekvenciájú WAV formátumban tárolok el.

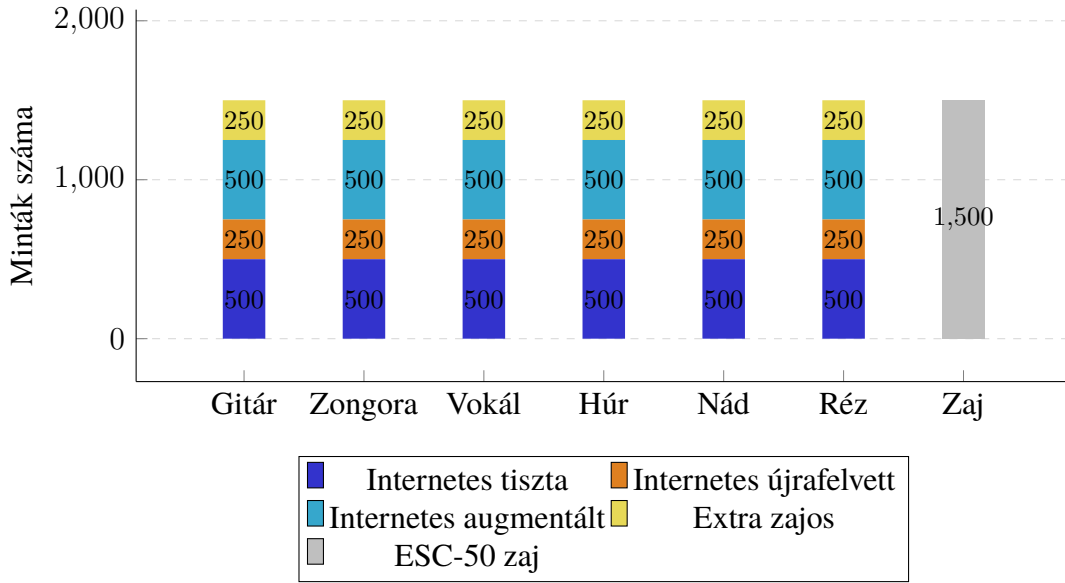
Az adathalmaz felépítésének és az adatfolyamnak a folyamatdiagramját a 4.1.4. ábra mutatja be.



4.1.4. ábra: A saját alapadatokból és adataugmentációból építkező hibrid adathalmaz folyamatdiagramja

A létrejött végző adathalmaz belső összetételét (az internetes tiszta, az internetes újrafelvett, az internetes augmentált és a tovább augmentált extra zajos minták, valamint az

ESC-50 zajminták pontos arányát) a 4.1.5. ábra szemlélteti.



4.1.5. ábra: A végső, kiegyensúlyozott hibrid adathalmaz belső összetétele kategóriák szerint

4.2. Jellemzőkinyerés tervezése

A nyers audiojelek nem alkalmasak közvetlenül a konvolúciós neurális hálózat bemeneteként, mivel a hullámforma (waveform) időtartománybeli ábrázolása túl magas dimenziójú és nehezen tanulható mintázatokot tartalmaz. Emiatt szükséges egy megfelelő 2D-s spektrális jellemzőkinyerési eljárás kiválasztása. A tervezés során három elterjedt módszert vizsgáltam meg a librosa [7] könyvtár implementációira építve: a rövid idejű Fourier-transzformációt (STFT), a Mel-frekvenciás kepsztrális együttartókat (MFCC), valamint a Log-Mel spektrogramot. Mindhárom jellemzőtípus azonos ablakozási paraméterekkel készül:

- FFT ablakméret: $n_{fft} = 1024$ minta,
- Ugrási hossz (hop length): $hop_length = 512$ minta,
- Mintavételi frekvencia: $SR = 16\,000$ Hz.

1 másodperces audioszelet esetén ($16\,000$ minta, $hop_length = 512$) az időtengely mérete: $\lfloor 16000/512 \rfloor + 1 = 32$ időkeret.

4.2.1. Log-Mel spektrogram

A Log-Mel spektrogram előállításának lépései:

1. Mel-szűrőbank alkalmazása $n_{mels} = 64$ sávval az STFT kimenetére,
2. Teljesítmény-decibelre konvertálás: $S_{dB} = 10 \cdot \log_{10}(S_{mel}/S_{ref})$.

Az eredmény egy 64×32 méretű mátrix, amelyet a CNN egysatornás képként ($64 \times 32 \times 1$) kap meg. A Log-Mel spektrogram előnye, hogy a Mel-skála a hallórendszer nemlineáris frekvenciaérzékelését modellezi, így a hangszerek jellegzetes formánsai és felharmonikusai hatékonyan jelennek meg.

4.2.2. STFT spektrogram

Az STFT (Short-Time Fourier Transform) spektrogram az amplitúdó-spektrum decibelre konvertált változata:

$$X_{dB}(f, t) = 20 \cdot \log_{10} \left(\frac{|X(f, t)|}{X_{ref}} \right)$$

Az eredmény egy 513×32 méretű mátrix ($n_{fft}/2 + 1 = 513$ frekvenciasáv). Ez a reprezentáció a teljes lineáris frekvenciaskálát megtartja.

4.2.3. MFCC (Mel-Frequency Cepstral Coefficients)

Az MFCC-t $n_{mfcc} = 40$ együtthatóval számolom. Az MFCC a spektrális burkoló tömör reprezentációja; az eredmény egy 40×32 méretű mátrix. Az MFCC jellemzőkre Z-score normalizálást alkalmazok sávonként:

$$\hat{x}_i = \frac{x_i - \mu_i}{\sigma_i + \epsilon}$$

ahol μ_i és σ_i az i -edik MFCC sáv átlaga és szórása, $\epsilon = 10^{-10}$.

4.2.4. A megfelelő jellemzőkinyerési eljárás kiválasztása

A három vizsgált jellemzőtípus (STFT, MFCC, Log-Mel) alapos elemzése után a következő szempontokat mérlegeltem a neurális hálózat bemenetének kiválasztásakor:

- Az **STFT spektrogram** előnye, hogy a teljes lineáris frekvenciaspektrumot megőrzi, azonban a 513×32 -es méret túl nagy dimenziójú bemenetet eredményezne. Ez nemcsak jelentősen növelné a hálózat paraméterszámát és a tanítási időt, de a magas frekvenciás sávok felesleges részletgazdagsága miatt a túlilleszkedés (overfitting) kockázatát is növelné.
- Az **MFCC** bár rendkívül kompakt (40×32), a diszkrét koszinusz transzformáció (DCT) alkalmazásával eldobja a spektrális finomszerkezetet és a felharmonikusok közötti összefüggéseket. Ez beszédfelismerésnél előnyös (ahol a hangmagasságtól független fonémákat keressük), de a zenei hangszerek osztályozásánál a felharmonikusok aránya hordozza a hangszín lényegi információját, így az MFCC használata információvesztéssel járna.
- A **Log-Mel spektrogram** (64×32) jelenti a legjobb kompromisszumot. A Mel-skála nemlineárisan sűríti a frekvenciatengelyt az emberi hallásnak megfelelően (finomabb

felbontás a mély tartományban, durvább a magasban), így megőrzi a hangszerek hangszínének felismeréséhez szükséges felharmonikus-szerkezetet, miközben alacsonyan tartja a bemeneti dimenziót.

A fenti szempontok alapján a valós idejű hangszerfelismerő rendszerem elsődleges jellemzőjének a **Log-Mel spektrogramot** választottam, a konvolúciós neurális hálózat architektúráját és a valós idejű feldolgozást is erre a bemenetre terveztem. A három vizsgált jellemző legfontosabb tulajdonságait a 4.2.1. táblázat foglalja össze.

4.2.1. táblázat: A három vizsgált jellemzőkinyerési módszer összehasonlító elemzése

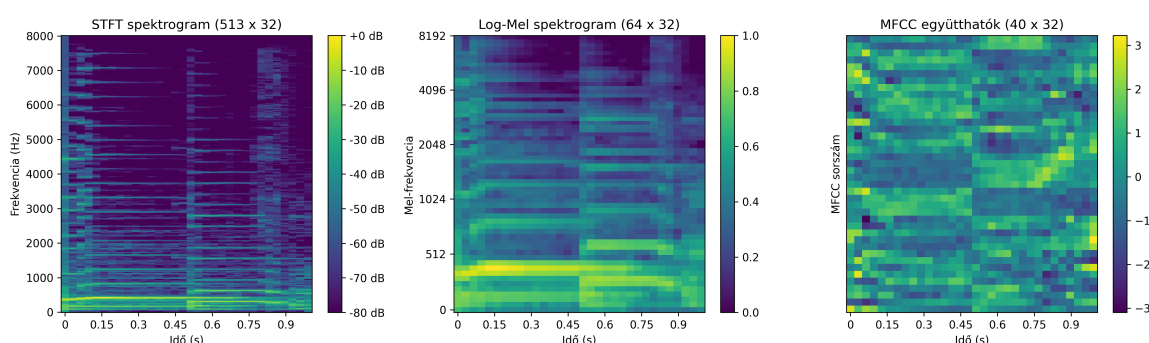
Jellemző	Dimenzió	Frekvencia-skála	Fő előny	Fő hátrány
STFT	513×32	Lineáris	Megőrzi a teljes spektrális finomszerkezetet.	Túl nagy dimenziószám, növeli a túlilleszkedés kockázatát.
Log-Mel	64×32	Logaritmikus Mel-skála	Az emberi hallást modellezi; megőrzi a felharmonikusokat.	Enyhe információvesztés a frekvenciatömörítés miatt.
MFCC	40×32	Kepsztrális	Rendkívül kompakt és dekorrelált reprezentáció.	Diszkrét koszinusz transzformáció miatt eldobja a felharmonikusokat.

4.2.5. Normalizálás

A kiválasztott Log-Mel jellemzőket globális min-max normalizálással skálázom a $[0, 1]$ intervallumra:

$$\hat{X} = \frac{X - X_{min}}{X_{max} - X_{min} + \epsilon}$$

Ez biztosítja, hogy a CNN bemenete konzisztens értéktartományban maradjon.



4.2.1. ábra: A három vizsgált jellemzőtípus vizualizációja ugyanazon gitárminta esetén: (a) STFT spektrogram (513×32), (b) normalizált Log-Mel spektrogram (64×32), (c) normalizált MFCC együtthatók (40×32)

4.2.6. Jellemzőkinyerés kimeneti állományai

A jellemzőkinyerési folyamat sikeres lefutása után a kinyert mátrixok és a hozzájuk tartozó osztálycímkék a projekt gyökerében lévő `processed_data/` könyvtárba kerülnek elmentésre NumPy formátumban (`.npy` kiterjesztéssel). A generált fájlok a következők:

- **Log-Mel spektrogram:** `X_log_mel_full.npy` (a jellemzőmátrixokat tartalmazó tömb) és `y_log_mel_labels.npy` (a hangszerosztályok sorszámaikat tartalmazó címketömb),
- **STFT spektrogram:** `X_stft_full.npy` és `y_stft_labels.npy`,
- **MFCC együtthatók:** `X_mfcc_full.npy` és `y_mfcc_labels.npy`.

A normalizált és binárisan tömörített fájlformátum biztosítja, hogy a tanítási fázisban a minták betöltése rendkívül gyorsan, közvetlenül a memóriába (RAM) történhessen meg, elkerülve a felesleges lemezműveleteket.

4.3. Adataugmentáció tervezése

A modell valós környezetben való robusztusságának javítása érdekében minden hangszermintát (a noise osztályt kivéve) egy augmentált változattal egészítjük ki. Az augmentáció a fogyasztói mikrofonos felvétel körülményeit szimulálja, ezért a belső elnevezése `apply_macbook_augment`. Az alkalmazott dataaugmentációs lánc felépítését és a lépések sorrendjét a 4.3.1. ábrán mutatom be, míg az augmentáció hatását a spektrális jellemzőkre (egy gitárminta példáján keresztül) a 4.3.2. ábra szemlélteti.

Az augmentációs lánc a következő lépésekből áll:

4.3.1. Szobai reverb szimuláció

A reverb szimulációt 80%-os valószínűséggel alkalmazom. A szoba akusztikáját egy egyszerű FIR visszhang-moddal közelítem, amely négy visszaverődési pontot (tap) használ:

- Késleltetés (*delay*): véletlenszerű, 15–40 ms között,
- Csillapítási tényező (*decay*): véletlenszerű, 0,2–0,45 között,
- Visszaverődési pontok: d , $1,5d$, $2,1d$, $2,8d$ mintányi késleltetésnél,
- Csillapítás: $decay^1$, $decay^2$, $decay^3$, $decay^4$.

4.3.2. Frekvenciaválasz-torzítás (Pre-emphasis)

A mikrofon és a szoba együttes frekvenciaválaszának szimulálására pre-emphasis szűrőt alkalmazok, amelynek együtthatója véletlenszerűen 0,92 és 0,98 között változik:

$$y[n] = x[n] - \alpha \cdot x[n - 1]$$

4.3.3. Környezeti zajkeverés (Noise mixing)

A háttérzajt az ESC-50 adathalmaz [1] véletlenszerűen kiválasztott zajmintájával keverem a hangszermintához. A keverési arány (SNR) véletlenszerűen 5–15 dB között változik:

$$y_{mixed} = x_{clean} + \sqrt{\frac{P_{clean}}{SNR_{lin} \cdot P_{noise}}} \cdot x_{noise}$$

ahol $SNR_{lin} = 10^{SNR_{dB}/10}$, és P a jel átlagos teljesítménye.

4.3.4. Szoftklip torzítás (Soft clipping)

A mikrofon bemeneti erősítőjének telítési karakterisztikáját hiperbolikus tangens szoftklippel modellezem:

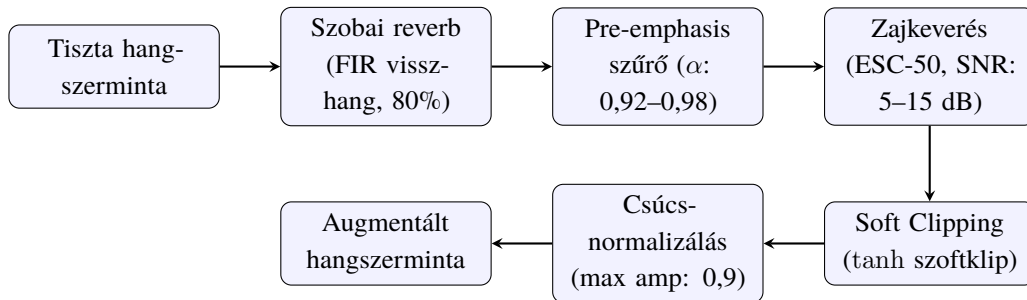
$$y[n] = \frac{\tanh(drive \cdot x[n])}{\tanh(drive)}$$

ahol $drive$ véletlenszerűen 2,0 és 4,0 között változik.

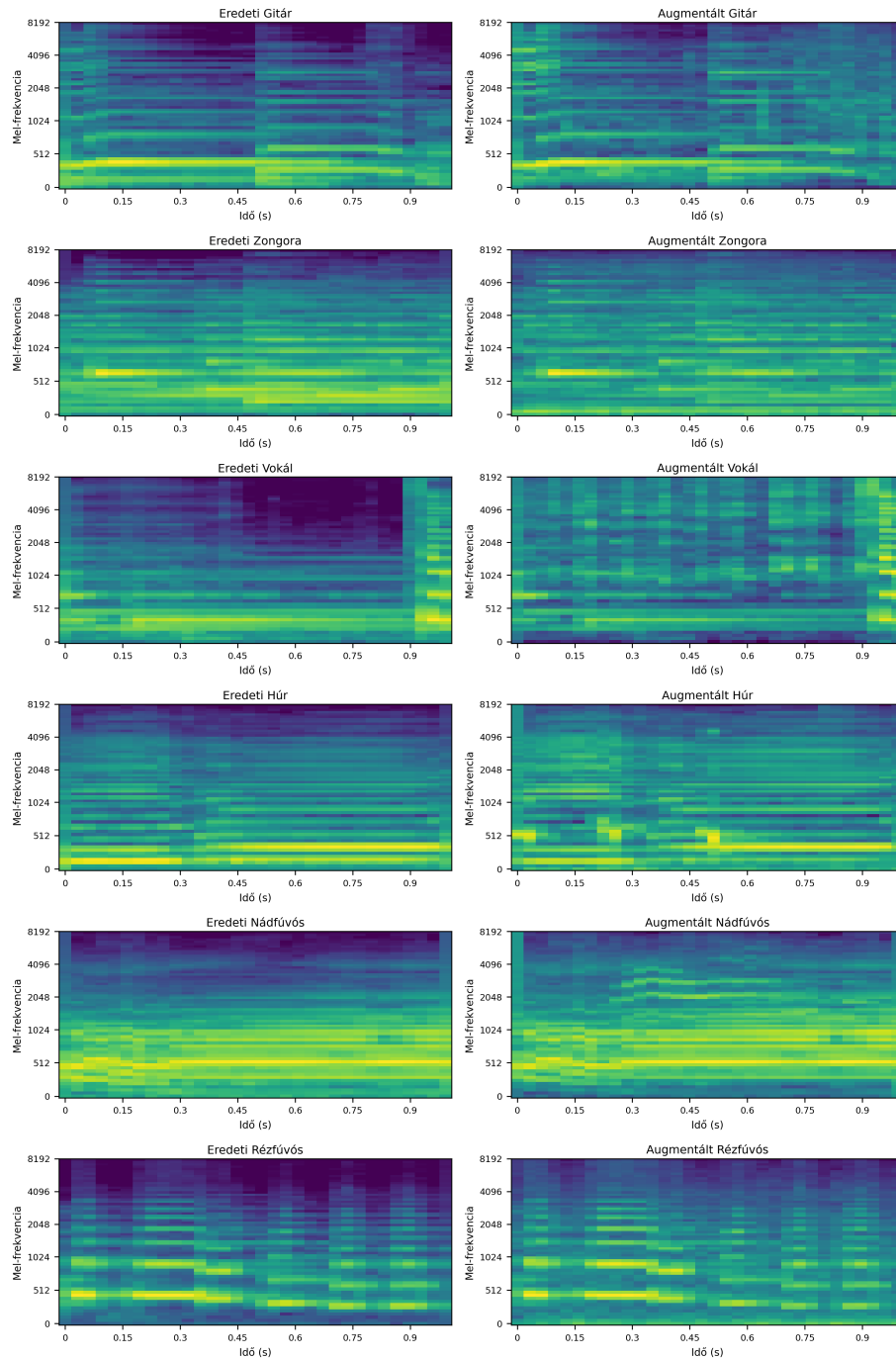
4.3.5. Végső normalizálás

Az augmentált jelet csúcscsértékre normalizálom, hogy az amplitúdó ne haladja meg a 0,9-es szintet:

$$y_{norm} = \frac{y}{\max(|y|)} \cdot 0,9$$



4.3.1. ábra: A MacBook/mikrofonos felvételi körülményeket szimuláló adataugmentációs folyamatlánc



4.3.2. ábra: A hat hangszerosztály (gitár, zongora, vokál, hegedű, klarinét, kürt) egy-egy reprezentatív mintájának normalizált Log-Mel spektrogramja az eredeti (tiszt) és az adataugmentált (MacBook/mikrofon szimulációs láncsal torzított) állapotban

Az augmentáció eredményeként az alapadathalmaz mérete megduplázódik a 6 hangszerosztály esetén (a noise osztályt nem augmentálok), így alakítva ki a végső adathalmazt: $6 \times 750 \times 2 + 1500 = 10\,500$ minta.

Az augmentációs eljárás helyességének és minőségének manuális ellenőrzése, illetve összehasonlítása céljából a rendszer minden hangszerosztályból kiment néhány bemutató mintát (az eredeti és az augmentált változat párosait) az `augmented_samples/` könyvtárba. Ez a gyakorlatban lehetővé teszi a fejlesztő számára, hogy mind füllel (hallás útján), mind

vizuálisan meggyőződjön arról, hogy az alkalmazott szimulációs torzítások (például visszhang és zajkeverés) valóságosak, és nem teszik felismerhetetlenné a hangszerek alapvető karakterét.

4.4. A 2D CNN hálózat architektúrájának tervezése

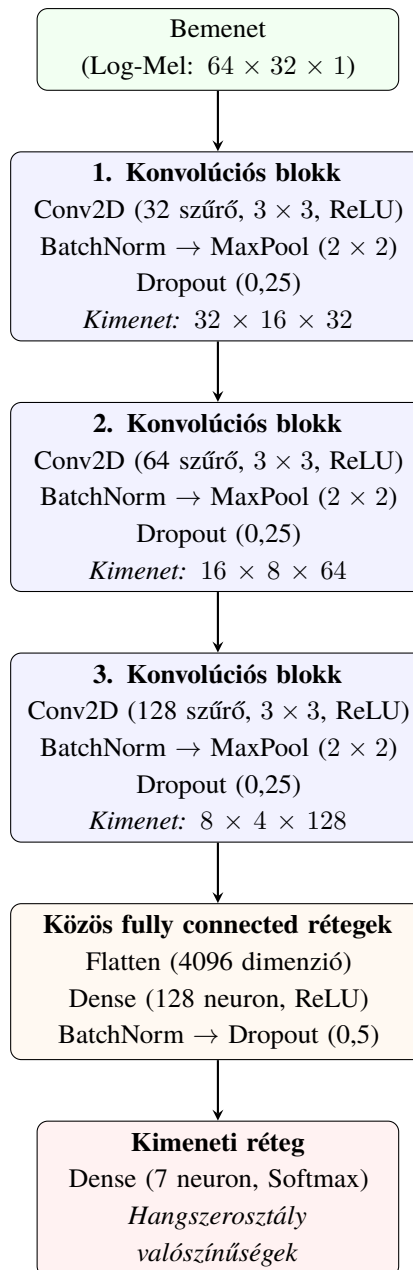
A tervezett hangszerfelismerő rendszer magját egy mély konvolúciós neurális hálózat (CNN) alkotja, amely a Log-Mel spektrogramok 2D-s vizuális mintázatait tanulja meg osztályozni. Ebben a fejezetben részletezem a hálózat felépítését, a rétegek sorrendjét és paramétereit, valamint a hálózat betanításához használt konfigurációkat. A tervezett hálózati rétegek specifikációit a 4.4.1. táblázat foglalja össze, a rétegek közötti adatfolyamot és az architektúra grafikus felépítését pedig a 4.4.1. ábra ábrázolja.

4.4.1. A hálózati architektúra részletei

A rendszer magját egy háromblokkos konvolúciós hálózat (2D CNN) alkotja. Minden blokk a következő rétegekből áll: 2D konvolúció → Batch Normalization → Max Pooling → Dropout. A hálózat bemenete a kiválasztott, normalizált Log-Mel spektrogram ($64 \times 32 \times 1$).

4.4.1. táblázat: A tervezett 2D CNN architektúra rétegei (Log-Mel bemenet: $64 \times 32 \times 1$)

Réteg	Szűrők / Neuronok	Kimenet	Paraméterek
Conv2D	32 szűrő, 3×3 , ReLU, same	$64 \times 32 \times 32$	320
BatchNorm	–	$64 \times 32 \times 32$	128
MaxPool2D	2×2	$32 \times 16 \times 32$	0
Dropout	0,25	$32 \times 16 \times 32$	0
Conv2D	64 szűrő, 3×3 , ReLU, same	$32 \times 16 \times 64$	18 496
BatchNorm	–	$32 \times 16 \times 64$	256
MaxPool2D	2×2	$16 \times 8 \times 64$	0
Dropout	0,25	$16 \times 8 \times 64$	0
Conv2D	128 szűrő, 3×3 , ReLU, same	$16 \times 8 \times 128$	73 856
BatchNorm	–	$16 \times 8 \times 128$	512
MaxPool2D	2×2	$8 \times 4 \times 128$	0
Dropout	0,25	$8 \times 4 \times 128$	0
Flatten	–	4096	0
Dense	128, ReLU	128	524 416
BatchNorm	–	128	512
Dropout	0,5	128	0
Dense	7, Softmax	7	903
Összesen			~619 399



4.4.1. ábra: A tervezett 2D CNN hálózati architektúra felépítése és adatfolyama

4.4.2. Tanítási konfiguráció

A 2D CNN modell tanítása során alkalmazott konfigurációt és hiperparamétereket úgy válogattam meg, hogy biztosítsák a modell stabil konvergenciáját és elkerüljék a túlilleszkedést (overfitting), miközben hatékonyan kihasználják a rendelkezésre álló számítási erőforrásokat. A modell optimalizálását, a veszteségfüggvényt, az adathalmaz felosztását és a tanulási folyamatot szabályozó callback mechanizmusokat a 4.4.2. táblázat foglalja össze részletesen.

4.4.2. táblázat: A modell tanításához használt konfigurációs paraméterek és beállítások

Paraméter / Beállítás	Érték / Részletek
Optimalizáló	Adam (alapértelmezett tanulási ráta: 10^{-3})
Veszteségfüggvény	Sparse Categorical Crossentropy
Tanító/tesztelő felosztás	80% / 20% (rétegzett / stratified felosztásban)
Batch méret	32
Maximális epochszám	100
Early Stopping	val_loss monitorozás, patience = 7, a legjobb súlyok visszaállításával (restore_best_weights=True)
ReduceLROnPlateau	val_loss monitorozás, patience = 3, faktor = 0,5, minimális tanulási ráta: 10^{-6}
ModelCheckpoint	val_accuracy alapján a legjobb modell mentése .keras formátumban

A táblázatban szereplő beállítások elméleti háttere és működési célja a következő:

- **Adam optimalizáló:** Adaptív módon szabályozza a tanulási rátát minden egyes paraméternél, ami gyorsabb konvergenciát és jobb stabilitást eredményez a hagyományos sztochasztikus gradiens leereszkedéshez (SGD) képest.
- **Early Stopping callback:** Megakadályozza a túlilleszkedést (overfitting). Ha a validációs veszteség (val_loss) 7 egymást követő epochoz tartozóan már nem csökken, a tanítási folyamat automatikusan leáll, és visszaállítja a modellt a legjobb súlyokat tartalmazó állapotba.
- **ReduceLROnPlateau callback:** Lehetővé teszi a tanulási ráta menet közbeni finomhangolását. Ha a modell elakad egy lokális minimum közelében (azaz a val_loss nem csökken 3 epoch óta), a tanulási sebességet lefelezi (0,5-ös faktor), így segítve a precízebb konvergenciát.

A tanítás a Google Colab felhőszolgáltatás GPU-ján történik, az adatokat és a mentett modelleket a Google Drive-on tároljuk és onnan töltjük vissza a projektbe.

4.4.2.1. A tanítás során generált kimenetek

A tanítás lefutását követően a validációs adathalmazon legjobban teljesítő modellállapotok a projekt gyökerében lévő models/ könyvtárba kerülnek mentésre. A különböző jellemzőtípusok alapján a következő fájlok jönnek létre:

- **Log-Mel alapú CNN modell:** best_log_mel_2dcnn_model.keras,
- **STFT alapú CNN modell:** best_stft_2dcnn_model.keras,
- **MFCC alapú CNN modell:** best_mfcc_2dcnn_model.keras.

A TensorFlow .keras formátuma magában foglalja a hálózat architektúráját, a betanult súlymátrixokat, valamint az optimalizáló állapotát is. Ezen kimenetek közvetlenül betölthetők a `tf.keras.models.load_model` függvény segítségével, ami lehetővé teszi, hogy a valós idejű felismerő modul másodpercek alatt elinduljon a korábban elmentett legjobb modellállapot betöltésével.

4.5. A valós idejű felismerő modul tervezése

A valós idejű hangszerfelismerő modul célja, hogy a számítógép mikrofonján keresztül érkező folyamatos audiojelet valós időben dolgozza fel, és minimális késleltetéssel jelezze ki a felismert hangszert és a predikció konfidenciaértékét. A modul stabil, aszinkron működésének megvalósításához egy többszálú architektúrát és egy egyedi predikciós stabilizáló logikát terveztem.

4.5.1. Audio stream és aszinkron puffereelés

A valós idejű hangfelvétel és a jelfeldolgozási lépések szétválasztásához többszálú jelfeldolgozó láncot alkalmazok. A hangkártya fizikai bemenetének kezeléséért a `sounddevice` könyvtár `InputStream` API-ja felel, amely egy magas prioritású háttérszálon futó hardveres megszakítási visszahívásos (callback) mechanizmusra épül:

- **Mintavételi frekvencia (f_s):** 16 000 Hz,
- **Csatornaszám:** 1 (monó),
- **Lépésköz (Step duration):** 0,5 másodperc (8000 minta).

A beérkező audioadatok kezelése egy szálbiztos FIFO soron (`queue.Queue`) keresztül történik. Az audio callback függvény kizárólag a hardver puffereából érkező 8000 mintás csomagokat helyezi el a sorban, majd azonnal visszatér. Ez garantálja, hogy a CPU-igényes neurális hálózati következtetés és a GUI-frissítés nem blokkolja a hangkártya illesztőprogramját, elkerülve az audio puffer alulcsordulását (buffer underflow).

A fő feldolgozó szál folyamatosan olvassa a sorból a 0,5 másodperces blokkokat. A predikció alapját egy folyamatos, 1,0 másodperces (16 000 minta) csúszóablak képezi. A 50%-os átfedés (overlap) eléréséhez egy körpuffer-szerű logikát alkalmazok: minden új 0,5 másodperces blokk beérkezésekor a puffer tartalmát balra csúsztatom 8000 mintával, és az új adatot a puffer végére írom (roll művelet). Ezáltal a rendszer másodpercenként kétszer hajt végre predikciót az utolsó 1 másodpercnyi hanganyagra vonatkozóan.

4.5.2. Jellemzőkinyerés és normalizálás

Minden predikciós ciklusban az 1 másodperces pufferből kinyerem a Log-Mel spektrogram jellemzőket. A kinyert spektrális mátrix dimenziója 64×32 , ami megegyezik a tanítás során használt bemenettel.

A neurális hálózat bemeneti rétege $[0, 1]$ intervallumba skálázott adatokat vár. A valós idejű jellemzőkre a tanítási fázissal azonos min-max normalizálást alkalmazok decibel alapú korlátokkal:

$$X_{\text{norm}} = \frac{X_{\text{dB}} - \text{min_dB}}{\text{max_dB} - \text{min_dB}}$$

ahol $\text{min_dB} = -80,0$ dB és $\text{max_dB} = 0,0$ dB. A kapott értékeket a $[0, 1]$ intervallumra vágom le (clip művelet), majd a tenzort átformázom a $1 \times 64 \times 32 \times 1$ bemeneti formára (batch és csatorna dimenziók hozzáadásával), és átadom a betöltött 2D CNN modellnek.

4.5.3. Zajkapu és csenddetektálás

A valós idejű működés során fontos szempont, hogy ha nincs zenei bemenet (például csend van a szobában), a rendszer ne kíséreljen meg hangszerosztályt predikálni a mikrofon alapzaja alapján, elkerülve a téves pozitív detektálásokat. Ennek érdekében a jelfeldolgozási lánc elejére egy szoftveres zajkaput (Noise Gate) terveztem.

A fő feldolgozó szál a jellemzőkinyerés megkezdése előtt kiszámítja a csúszóablakban lévő 1 másodperces audiojel $x(n)$ RMS (Root Mean Square) energiáját:

$$E_{\text{RMS}} = \sqrt{\frac{1}{N} \sum_{n=0}^{N-1} x(n)^2}$$

ahol $N = 16\,000$ a mintaszám. A rendszer ezt az értéket összehasonlítja egy előre meghatározott zajkapu-küszöbértékkel (θ_{gate}):

- Ha $E_{\text{RMS}} < \theta_{\text{gate}}$ (ahol $\theta_{\text{gate}} = 0,005$), a bemenetet csendnek vagy háttérzajnak tekintjük. A rendszer ekkor kikerüli (bypass) a jellemzőkinyerési és a neurális hálózati következtetési fázist, és azonnal a Zaj/Csend osztályt jeleníti meg a kimeneten 100% konfidenciával.
- Ha $E_{\text{RMS}} \geq \theta_{\text{gate}}$, a jelfeldolgozási lánc normál módon folytatódik a Log-Mel jellemzők kinyerésével és a 2D CNN osztályozó futtatásával.

Ez a megoldás nemcsak a téves detektálásokat akadályozza meg csendes környezetben, hanem jelentősen csökkenti a CPU terhelését is azáltal, hogy inaktív időszakokban mentesíti a hardvert a neurális hálózat futtatásának igényétől.

4.5.4. Predikciós döntési logika és hiszterézis

A hangszerosztályok pillanatnyi valószínűségei a zenefájlokban és az élő játékban tapasztalható dinamikus változások (tranziensek, vibrato, környezeti zajok) miatt gyorsan ingadozhatnak. Ha közvetlenül az aktuális predikció legnagyobb valószínűségű osztályát jelenítenénk meg, a kijelzőn zavaró, gyors váltások („villogás”) jelennének meg. Ennek kiszűrésére egy kétszintű stabilizációs logikát terveztem:

1. **Temporális mozgóátlag simítás:** A modell kimeneti valószínűség-vektorait egy $W = 4$ elemű FIFO sorban tárolom (amely az utolsó 2 másodperc predikciós történetét fedi le). A döntéshozatal a simított valószínűség-vektor alapján történik:

$$\bar{p}_c^{(t)} = \frac{1}{W} \sum_{i=0}^{W-1} p_c^{(t-i)}$$

ahol $p_c^{(t-i)}$ a modell által a c osztályra becsült valószínűség a $(t - i)$ -edik időlépésben.

2. **Hiszterézis mechanizmus:** Annak érdekében, hogy megakadályozzuk a két közeli valószínűségű osztály közötti ugrálást (chattering), egy $H = 0,05$ (5%) értékű hiszterézis bónuszt adunk a jelenleg aktív (kijelzett) osztály $C^{(t-1)}$ simított valószínűségéhez a maximumkeresés előtt:

$$a_c^{(t)} = \bar{p}_c^{(t)} + H \cdot \mathbb{I}(c = C^{(t-1)})$$

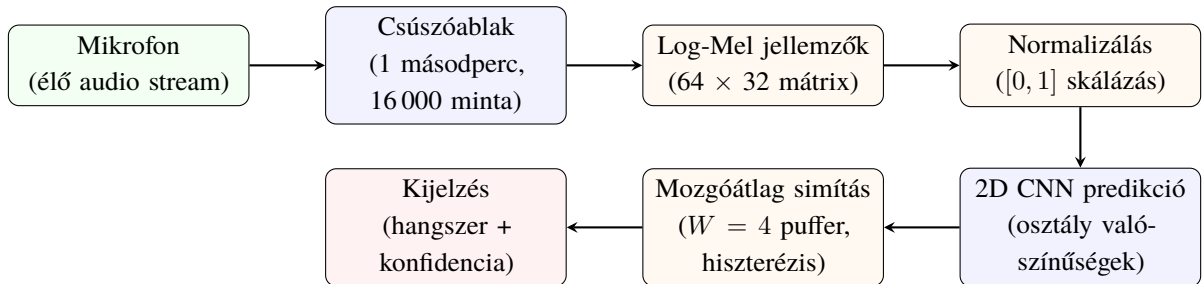
ahol $\mathbb{I}$ az indikátorfüggvény. A jelölt új osztályt a módosított vektor maximumhelye adja meg:

$$c_{\text{cand}} = \arg \max_c a_c^{(t)}$$

3. **Konfidencia-küszöb:** A kijelzett hangszercsere csak akkor történik meg, ha a jelölt új osztály simított valószínűsége (bónusz nélkül) eléri a $\tau = 0,3$ (30%) minimális konfidencia-küszöbértéket:

$$C^{(t)} = \begin{cases} c_{\text{cand}}, & \text{ha } c_{\text{cand}} \neq C^{(t-1)} \text{ és } \bar{p}_{c_{\text{cand}}}^{(t)} \geq \tau \\ C^{(t-1)}, & \text{egyébként} \end{cases}$$

Ez a tervezési megoldás rendkívül stabil vizuális visszajelzést biztosít a felhasználói felületen, miközben az átfedéseknek köszönhetően a fizikai reakcióidő legfeljebb 0,5–1,0 másodperc marad. A valós idejű felismerési folyamat egyes lépéseit, a bemeneti audio stream rögzítésétől a konvolúciós hálózat predikcióján és a hiszterézises simításon át a kijelzésig a 4.5.1. ábra foglalja össze vizuálisan.

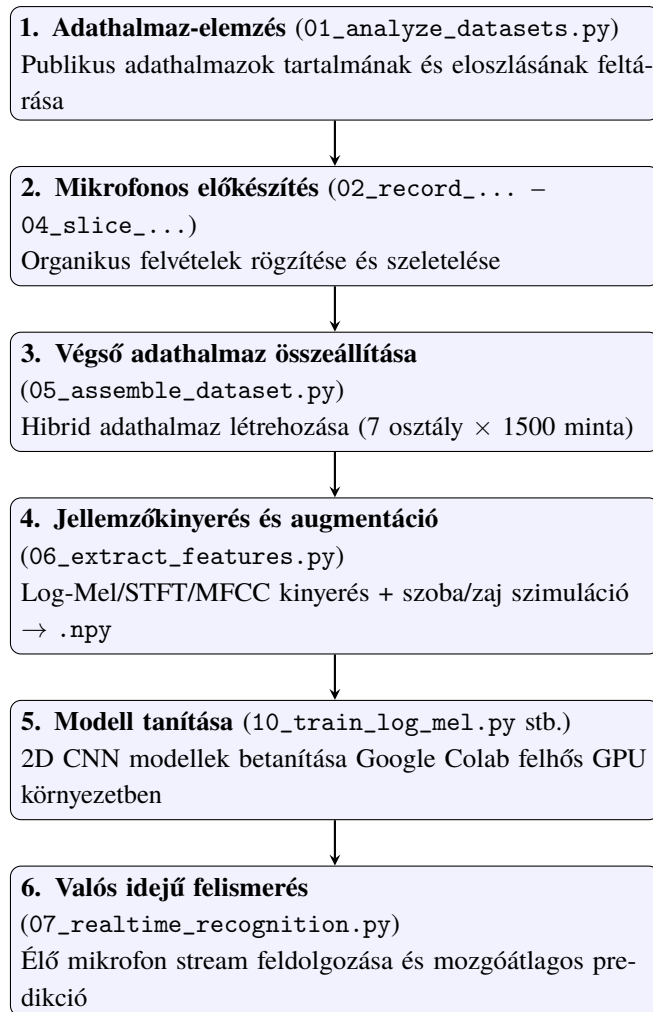


4.5.1. ábra: A valós idejű felismerési folyamat lépései az élő hangbeviteltől a kijelzésig

4.6. A feldolgozási folyamat összefoglaló áttekintése

A teljes rendszer a tervezéstől a megvalósításig egymásra épülő fázisokból áll, amelyek a teljes adatfeldolgozási és tanítási láncot lefedik. Ezeket a tervezési fázisokat az alábbi számozott pontokban foglalom össze, a fázisok közötti összefüggéseket és a teljes rendszerfolyamatot pedig a 4.6.1. ábra szemlélteti.

1. **Adathalmaz-elemzés** (01_analyze_datasets.py): A 6 publikus adathalmaz kategóriaeloszlásának feltárása.
2. **Mikrofonos előkészítés** (02–04): Összefűzés, felvétel, szeletelés.
3. **Végső adathalmaz összeállítása** (05_assemble_dataset.py): 7 osztály $\times$ 1500 minta.
4. **Jellemzőkinyerés és augmentáció** (06_extract_features.py): Log-Mel, STFT, MFCC kinyerése + MacBook-augmentáció $\rightarrow$.npy fájlok.
5. **Modell tanítása** (10_train_* szkriptek): Google Colab GPU-n.
6. **Valós idejű felismerés** (07_realtime_recognition.py): Élő mikrofon bemenet.



4.6.1. ábra: A hangszerfelismerő rendszer teljes feldolgozási folyamatának lépései

5. ÜZEMBE HELYEZÉS ÉS KÍSÉRLETI EREDMÉNYEK

Ez a fejezet bemutatja a rendszer üzembe helyezésének gyakorlati lépéseit, a szoftver fejlesztése és futtatása során felmerült technikai problémákat és azok megoldásait, valamint a betanított modellekkel végzett kísérletek és mérések számszerűsített eredményeit.

5.1. Üzembe helyezési lépések

A hangszerfelismerő rendszer két fő részből áll: a háttérben futó gépi tanulási és jelfeldolgozó modulokból, valamint a valós idejű parancssori felhasználói alkalmazásból (CLI Dashboard). A szoftver üzembe helyezéséhez az alábbi lépések szükségesek:

1. **Python környezet előkészítése:** A futtatáshoz legalább Python 3.10 verzió szükséges. Ajánlott egy izolált virtuális környezet (virtual environment) létrehozása a függőségek kezelésére:

```
python3 -m venv venv
source venv/bin/activate
```

2. **Rendszerszintű függőségek telepítése:** A valós idejű hangrögzítésért felelős `sounddevice` könyvtár a PortAudio C-könyvtárra épül. macOS rendszereken ezt a Homebrew segítségével kell telepíteni:

```
brew install portaudio
```

Windows és Linux platformokon a csomagkezelők többsége automatikusan fordítja vagy tartalmazza a szükséges binárisokat.

3. **Python könyvtárak telepítése:** A szükséges csomagok listáját a `requirements.txt` tartalmazza. Telepítésük a következő paranccsal végezhető el:

```
pip install -r requirements.txt
```

A legfontosabb telepített csomagok: `tensorflow` (neurális hálózat futtatása), `librosa` (jellemzőkinyerés), `sounddevice` (mikrofonkezelés), valamint a vizualizációért felelős `matplotlib` és `numpy`.

4. **Modellfájlok elhelyezése:** A betanított neurális hálózati modelleket a `best_log_mel_2dcnn_model.keras` és a `best_stft_2dcnn_model.keras` fájlok formájában a projekt `models/` alkönyvtárába kell elhelyezni.

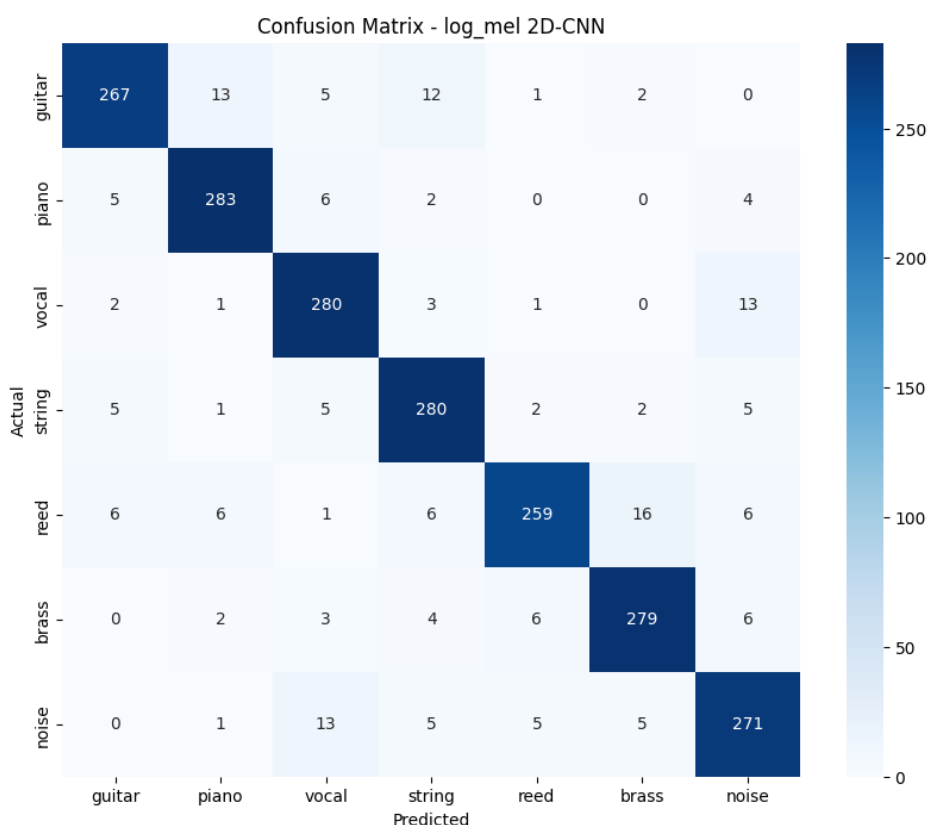
5. **Alkalmazás indítása:** A valós idejű parancssori felület az alábbi parancs futtatásával indítható el:

```
python scripts/07_realtime_recognition.py
```

5.2. Felmerült problémák és megoldások

5.3. Kísérleti eredmények és mérések

A legjobb eredményt nyújtó Log-Mel alapú modell hangszerenkénti osztályozási mátrixát (konfúziós mátrix) vizsgáltam meg részletesebben. A 5.3.1. ábra hő térkép (heatmap) formájában mutatja a hangszerosztályok közötti tévesztési arányokat és a pontos mintaszámokat az osztályozás során.



5.3.1. ábra: A Log-Mel spektrogram alapú 2D CNN modell konfúziós mátrixának hő térképe

A konfúziós mátrix hő térképéből leolvasható, hogy a legkönnyebben és legbiztosabban felismerhető kategória a Zongora (94,3%, azaz 283 helyes predikció a 300 tesztmintából), valamint az Ének és a Vonósok (mindkettő 93,3%, azaz 280/300 helyes osztályozás). A zongora tiszta leütési tranzienzei és a vonósok egyenletes, hosszan tartó harmonikus felhangszerkezete jól elkülöníthető vizuális mintázatot adnak a Log-Mel spektrogramon, miközben az énekhang egyedi formánsai (a vokális traktus rezonanciája) szintén markáns ujjlenyomatként jelennek meg.

A legnagyobb arányú tévesztések a következők szerint alakultak:

- **Fúvós hangszerek:** A Nádas fúvósok mintáinak 5,3%-át (16 mintát) a modell Rézfúvósnak osztályozta. Ez a tévesztés akusztikailag rendkívül indokolt, mivel például az egynás szaxofon (nádas fúvós) és a trombita (rézfúvós) felhangszerkezete és megszólalási tranziensei bizonyos hangmagasságoknál nagyon hasonlóak.
- **Zongora és Gitár:** A Gitár minták 4,3%-át (13 mintát) a modell Zongorának osztályozta. Ez a pengetett és leütött húrok fizikai rezgésének kezdeti lecsengési hasonlóságából fakad, különösen az akusztikus gitár esetében.
- **Ének és Háttérzaj:** A Háttérzaj minták 4,3%-át (13 mintát) Éneknek, míg az Ének minták 4,3%-át (13 mintát) Háttérzajnak jelölte a hálózat. Ez a mikrofonos zörejek, zihálások és az emberi beszéd/ének frekvenciatartományának átfedései miatt fordul elő, valamint a halkabb énekszakaszok és a környezeti zajok hasonlóságából adódik.

Mintent összevetve a Log-Mel alapú 2D CNN modell kiválóan teljesít: a legnehezebbnek bizonyuló Nádas osztályban is eléri a 86,3%-os pontosságot, míg a többi zenei és környezeti kategóriában 89% feletti egyedi pontosságot biztosít.

6. A RENDSZER FELHASZNÁLÁSA

Ez a fejezet az elkészült hangszerfelismerő rendszer gyakorlati alkalmazását mutatja be a végfelhasználó szempontjából. Részletesen ismertetem a szoftver indításának menetét, a terminál alapú interaktív felhasználói felület (CLI Dashboard) felépítését és színkódolását, valamint bemutatok több tipikus valós idejű működési forgatókönyvet (tesztesetet) szöveges konzol-mockupokon keresztül.

6.1. Az alkalmazás indítása és konfigurációja

A rendszer tervezése során kiemelt szempont volt a könnyű hordozhatóság és az alacsony erőforrás-igény. Mivel a szoftver egy interaktív, valós idejű parancssori felületen fut, nincs szükség erőforrás-igényes grafikus ablakkezelő rendszerek betöltésére, így az alkalmazás akár szervereken vagy korlátozott teljesítményű beágyazott eszközökön is futtatható.

Az indítás előfeltételei az alábbiak:

1. A virtuális környezet aktiválása a projekt gyökérkönyvtárban:

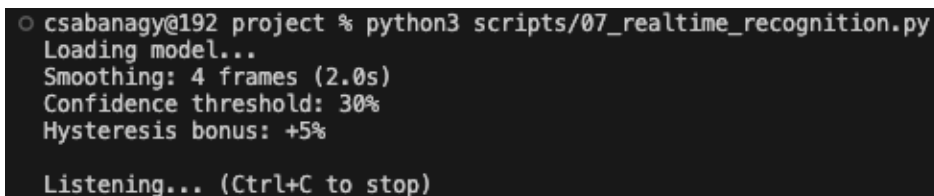
```
source venv/bin/activate
```

2. A külső mikrofon vagy a beépített hangbemeneti eszköz csatlakoztatása és alapértelmezetté tétele a rendszerbeállításokban.

Az indítás a következő parancs futtatásával történik:

```
python scripts/07_realtime_recognition.py
```

Az alkalmazás indításakor a konzolon megjelennek a konfigurációs paraméterek, a betanított Keras modell betöltésének állapota, valamint az élő jelfeldolgozási folyamat megkezdését jelző üzenet (lásd a 6.1.1. ábrát):



```
csabanagy@192 project % python3 scripts/07_realtime_recognition.py
Loading model...
Smoothing: 4 frames (2.0s)
Confidence threshold: 30%
Hysteresis bonus: +5%

Listening... (Ctrl+C to stop)
```

6.1.1. ábra: Az alkalmazás indításakor megjelenő konfigurációs és betöltési paraméterek

6.2. A parancssori felhasználói felület felépítése

A valós idejű visszajelzést egy dinamikusan frissülő terminálsor (CLI Dashboard) biztosítja. A hagyományos, soronként görgető konzolos kiírás helyett a program a kocsivissza

(\r) karaktert használja, így a terminál egyetlen sorát írja felül folyamatosan a legfrissebb információkkal. Ez rezponzív, villódzásmentes felhasználói élményt nyújt.

A kijelző sor három fő vizuális komponensből áll:

1. **Dinamikus konfidencia-sáv:** Egy 20 karakter hosszúságú grafikus sáv, amely a felismert hangszer valószínűségét ábrázolja vizuálisan. A teli blokk (■) karakterek a konfidencia nagyságát mutatják, míg a satírozott (░) karakterek a fennmaradó részt.
2. **Aktív osztály és konfidencia érték:** A felismert hangszer megnevezése és a kiszámított valószínűségi százalékos érték (tizedes pontossággal).
3. **Másodlagos osztályok valószínűségei:** A többi hangszerosztály rövidített jelölése (G: Gitár, P: Zongora, V: Vokál, S: Vonós, R: Nádfúvós, B: Rézfúvós, N: Zaj/Csend) és azok pillanatnyi valószínűségi értékei, amelyek segítenek a felhasználónak átlátni a modell bizonytalansági tényezőit és az átfedéseket.

6.2.1. ANSI színkódolás és jelölések

A felület a terminál ANSI escape szekvenciáit kihasználva színesben jeleníti meg a felismert hangszereket. Minden hangszerosztály egyedi, kontrasztos színt kapott, így a felhasználó akár távolabbról, a szöveg elolvasása nélkül is azonnal látja az aktív hangszert. A színkódokat, jelöléseket és az osztályok leírását a 6.2.1. táblázat foglalja össze:

6.2.1. táblázat: A parancssori felületen alkalmazott ANSI színkódok és rövidítések kategóriáinként

Kategória	Terminál színe	Rövidítés	Leírás
guitar	Sárga	G	Pengetős hangszerek (pl. akusztikus gitár)
piano	Inverz Fekete/Fehér	P	Billentyűs hangszerek (zongora)
vocal	Ciánkék	V	Emberi énekhang (vokál)
string	Sötétvörös	S	Vonós hangszerek
reed	Narancssárga	R	Nádas fúvós hangszerek (szaxofon)
brass	Világosbarna	B	Rézfúvós hangszerek
noise	Szürke	N	Környezeti zaj / csend

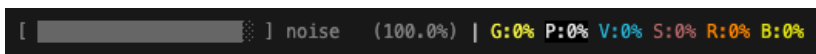
6.3. Működési forgatókönyvek és tesztesetek

Az alábbiakban bemutatom, hogyan reagál a konzolos interfész a különböző akusztikai bemenetekre valós időben.

6.3.1. 1. teszteset: Csendes szobai környezet (alapállapot)

Amikor a szobában nincs aktív hangszeres játék, csak a számítógép ventilátora vagy távoli környezeti zörejek hallatszanak, az RMS zajkapu átengedi a jelet (vagy a modell zajként

azonosítja), így a szürke színű noise osztály válik aktívvá kiugróan magas konfidenciával. A terminál kijelzése ekkor a 6.3.1. ábrán látható:



6.3.1. ábra: A rendszer kimenete csendes szobai környezet (zaj) esetén

6.3.2. 2. tesztet: Akusztikus gitárjáték (monofón dallam)

Amikor a felhasználó megszólaltat egy akusztikus gitárt a mikrofon közelében, a modell az első ablak után azonnal reagál. A simító ablaknak (2,0 másodperc) és a hiszterézisnek köszönhetően a kijelzés stabilan átvált sárga színű guitar felírra, miközben a többi hangszer valószínűsége minimálisra csökken (lásd a 6.3.2. ábrát):



6.3.2. ábra: A rendszer kimenete akusztikus gitárjáték érzékelésekor

6.3.3. 3. tesztet: Emberi énekhang (vokál)

Amikor a felhasználó énekelni vagy beszélni kezd a mikrofonba, a kijelző azonnal ciánkékre vált, és a vocal osztály jelenik meg. Mivel az ének felhangjai nagyon tiszták és erősek, a konfidencia érték gyorsan emelkedik (lásd a 6.3.3. ábrát):



6.3.3. ábra: A rendszer kimenete emberi énekhang (vokál) érzékelésekor

6.3.4. 4. tesztet: Szaxofon játék (átmeneti bizonytalanság)

A konfúziós mátrix elemzésénél láttuk, hogy a nádfúvósok és rézfúvósok között a legmagasabb a tévesztési arány. Amikor a felhasználó szaxofonon játszik, a modell kezdetben a rézfúvós és nádfúvós osztályok között oszcillálhat. A hiszterézis bónusz (+5%) és a konfidencia-küszöb (30%) megakadályozza a kijelző gyors villogását: a kijelző narancssárga reed színben marad mindaddig, amíg a rézfúvós alternatíva át nem lépi a biztonsági küszöböt (lásd a 6.3.4. ábrát):



6.3.4. ábra: A rendszer kimenete szaxofon játék (reed) érzékelésekor

6.3.5. Az alkalmazás leállítása

A felhasználó bármikor megszakíthatja az élő elemzést a terminálban kiadott Ctrl+C billentyűkombinációval. Ekkor a parancssori visszahívási szálak lezárulnak, a mikrofonbeme-

net felszabadul, és a program tiszta állapotban kilép a terminálba:

```
Stopped .
```

7. KÖVETKEZTETÉSEK

7.1. Megvalósítások

Megtervezni és megközelíteni egy, a cél szempontjából tökéletes, alacsony mintaszámú zenei adathalmazt szinte lehetetlen. Azonban egy igazán jót is kihívás, nem beszélve az osztályozó modell és a jellemzőkinyerő megválasztásáról és integrálásáról. Számomra igazi kihívást jelentett a rendszer élesítése. Már csak a minták kiválogatása alatt rengeteg szép hangszeres játékkal találtam szemben magam, ami csak emelte a munka iránti ambíciómat. Bár egy ilyen projekt sosincs kész, mindig van, amit javítani rajta, az eredmény minden kisebb-nagyobb hibája ellenére elégedettséggel tölt el, és büszkén foglal helyet a portfóliómban.

A munka során elért legfontosabb elméleti és gyakorlati megvalósítások az alábbi pontokban foglalhatók össze:

- Egy olyan hibrid adathalmaz létrehozása és pontos struktúrájának kialakítása, amely egyszerű módon, bárki által újra felhasználható és bővíthető más hangszerfelismerési kutatásokban is.
- Egy moduláris feldolgozási csővezeték kiépítése, amely lehetővé teszi a különböző jellemzőkinyerési eljárások (STFT, Log-Mel, MFCC) és modellek könnyű cseréjét a rendszerben.
- Az egyszerű, offline (fájl alapú) osztályozáson túl egy olyan valós idejű hangszerfelismerő alkalmazás megvalósítása, amely közvetlen mikrofon-bemenetet használ.
- A valós idejű predikciót simító és stabilizáló logika implementálása (mozgóátlag-számítás), amely megbízhatóbbá és gördülékenyebbé teszi a kijelzést.
- A 2D CNN modell optimalizálása a gyorsabb és alacsonyabb késleltetésű futás elérése érdekében, biztosítva a zökkenőmentes kiértékelést egy átlagos asztali számítógépen.

7.2. Hasonló rendszerekkel való összehasonlítás

A legtöbb hasonló hangszerfelismerő rendszer a szakirodalomban elég komplex modellekre épül, mint a ResNet vagy a CRNN. Ezeket általában hatalmas adathalmazokon tesztelik (mint az IRMAS vagy az NSynth), és bár jól megbirkóznak a polifón zenékkal, de elképesztően nagy a számítási igényük, így egy átlagos számítógépen szinte esélytelen őket valós időben futtatni.

Az én rendszerem ehhez képest egy sokkal könnyebb 2D CNN modellel dolgozik, amit kifejezetten a 7 hangszerosztályra alakítottam ki, bár a betanításához még így is a Google Colab felhőalapú erőforrásaira volt szükség. A mikrofonos, valós idejű predikció és a csúszóablakos

simítás viszont már stabil és szinte azonnali felismerést ad helyben egy egyszerű laptopon is. Bár tény, hogy a bonyolult zenekari polifóniával szemben ez a modell inkább csak az egyedi hangokra és kisebb zajokra jó, cserébe viszont egyáltalán nem terheli le a gépet és azonnal válaszol, ami a gyakorlatban sokkal használhatóbbá teszi.

7.3. Továbbfejlesztési lehetőségek

A rendszerben számtalan továbbfejlesztési lehetőség adott. Elsősorban a kutatás továbbvihető a zajos monofón felismeréstől a tiszta polifón hangszerfelismerés irányába. Ebben az esetben a meglévő softmax aktivációs függvényt sigmoidra kellene cserélnünk a kimeneti rétegen, és többcímkes (multi-label) osztályozást kellene alkalmaznunk, hiszen egyszerre több hangszer is megszólalhat. Ehhez azonban az eddiginél még sokkal több adatra lenne szükség, amelyet szintén az internetről gyűjthet össze az ember, vagy mesterségesen, két különálló hangszer hangját egy szkript segítségével összekeverve állíthat elő.

Ha még nagyobb kihívásra vágyik valaki, el lehet merészkedni a nagy, zajos zenekarkig (orchesztrákig), amelyek a jelen kiváló mérnökeinek is komoly fejtörést okoznak. Minden fejlesztés alapja először úgylis egy stabil és megbízható adathalmaz létrehozása.

Amennyiben a koncepcióval nem a zene felé, hanem egy iparibb környezet irányába indulnánk el, az adathalmaz átszervezésével készíthető egy ipari gépek állapotát felmérő szenzor is. Ez képes lenne megmondani a hang alapján, ha például egy gépből fogyóban van a kenőolaj, ha élettelené vált egy körfűrés (cirkula), vagy egyéb, az ipari gépek állapotával kapcsolatos előrejelzéseket adhatna.

8. IRODALOMJEGYZÉK

- [1] Karol J. Piczak. “ESC: Dataset for Environmental Sound Classification”. *Proceedings of the 23rd ACM International Conference on Multimedia*. 2015, 1015–1018. old.
- [2] Juan J. Bosch, Jordi Janer, Ferdinand Fuhrmann és Perfecto Herrera. “A Comparison of Sound Models for Automatic Instrument Recognition in Polyphonic Music Signals”. *Proceedings of the 13th International Society for Music Information Retrieval Conference (ISMIR)*. 2012, 55–60. old.
- [3] Jesse Engel, Cinjon Resnick, Adam Roberts, Sander Dieleman, Douglas Eck, Karen Simonyan és Mohammad Norouzi. “Neural Audio Synthesis of Musical Notes with WaveNet Autoencoders”. *Proceedings of the 34th International Conference on Machine Learning (ICML)*. 2017, 1068–1077. old.
- [4] Vincent Lostanlen, Joakim Andén és Mathieu Lagrange. “Extended Playing Techniques Reconstruction with Scattering Transform”. *Proceedings of the 21st International Conference on Digital Audio Effects (DAFx)*. 2018.
- [5] Carmine-Emanuele Cella, Daniele Ghisi, Philippe Esling és tsai. “TinySOL: An open dataset of orchestra instrument sounds”. *Proceedings of the International Conference on Technologies for Music Notation and Representation (TENOR)*. 2020.
- [6] Emilia Romani, Francesco Reguto, Sergio Giraldo és Rafael Ramirez. “A Real-time System for the Evaluation of Violin Sound Quality”. *Proceedings of the Sound and Music Computing Conference (SMC)*. 2015.
- [7] Brian McFee, Colin Raffel, Dawen Liang, Daniel P Ellis, Matt McVicar, Eric Battenberg és Oriol Nieto. “librosa: Audio and music signal analysis in python”. *Proceedings of the 14th Python in Science Conference*. 2015, 18–25. old.
- [8] Martín Abadi és tsai. *TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems*. Software available from tensorflow.org. 2015. URL: <https://www.tensorflow.org/>.

9. FÜGGELÉKEK

A függelék tartalmazza az „Automatikus hangszerfelismerés zenefelvételek alapján” című államvizsga-szakdolgozat teljes forráskódját, az adathalmaz-struktúrát és a betanított modellfájlokat tartalmazó digitális adathordozó könyvtárszerkezetének leírását, valamint a felhasznált szoftverfüggőségek összegzését.

9.1. A projekt könyvtárszerkezete

A benyújtott digitális függelék (amely a GitHubon elérhető nyilvános Git adattárban¹ is megtalálható) az alábbi főbb könyvtárakat tartalmazza. A számozott szkriptek a feldolgozási csővezeték (pipeline) egymásra épülő lépéseit jelzik.

9.1.1. táblázat: A projekt legfelső szintű könyvtárszerkezete

Könyvtár / Fájl	Leírás
thesis/	A L ^A T _E X formátumú szakdolgozat forrásfájljai, stílusfájl (sapiencia-thesis.cls), irodalomjegyzék (references.bib) és a generált main.pdf.
scripts/ owndataset/	A hangszerfelismerő rendszer Python szkriptjei (részletezés alább). A manuálisan összeválogatott és kivágott internetes hangminták hangszerosztályonként külön almappákban, valamint az újrafelvett minták a recorded_from_mic/ almappában.
hybrid_dataset_ own_final/	A végleges, összeszerelt hibrid adathalmaz 7 osztálymappával (guitar, piano, vocal, string, reed, brass, noise), osztályonként 1500 db, 1 másodperces, 16 kHz-es WAV minta.
processed_data/	Az előállított jellemzőmátrixok NumPy formátumban: X_stft_full.npy, X_log_mel_full.npy, X_mfcc_full.npy és a közös címkefájl y_labels_full.npy.
models/	A betanított Keras modellfájlok: best_stft_2dcnn_model.keras, best_log_mel_2dcnn_model.keras, best_mfcc_2dcnn_model.keras.
augmented_samples/	Az augmentációs lánc bemutató mintái: minden hangszerosztályból 3-3 eredeti-augmentált páros WAV fájl ellenőrzés céljából.
dataset/	A letöltött nyilvános referencia-adathalmazok (IRMAS, NSynth, Medley-solos-DB, TinySOL, Good-sounds, ESC-50) az elemzéshez.

¹<https://github.com/nCsab/Instrument-Recognition>

9.2. A feldolgozási csővezeték szkriptjei

A rendszer teljes feldolgozási lánc indexelt Python szkriptekből áll, amelyek a tervezés sorrendjében futtathatók. Az alábbi táblázat leírja minden szkript feladatát.

9.2.1. táblázat: A feldolgozási csővezeték szkriptjei és feladataik

Szkript	Feladat
01_analyze_datasets.py	A nyilvános referencia-adathalmazok (IRMAS, NSynth stb.) szerkezetének és statisztikáinak feltárása, összehasonlító elemzés készítése.
02_mic_prep_concatenate.py	Az újrafelvételéhez szükséges hanganyag előkészítése: a kiválasztott minták összefűzése egyetlen hosszú WAV fájlba sávszó-szinkronizálással.
03_mic_record_audio.py	A laptop beépített mikrofonjával vagy mobiltelefon mikrofonjával történő visszajátszás és valós idejű rögzítés.
04_mic_slice_audio.py	A rögzített mikrofonos felvétel darabolása 1 másodperces szegmensekre a sávszó-szinkronizáció alapján.
05_assemble_dataset.py	A végleges hibrid adathalmaz összeállítása: a tiszta minták, az újrafelvételek és az augmentált minták egyesítése és randomizált másolása a hybrid_dataset_own_final/ könyvtárba.
05a_prepare_hybrid_generic.py	Kiegészítő előkészítő szkript a hibrid adathalmaz generikus felépítéséhez.
06_extract_features.py	A fő jellemzőkinyerő szkript, amely az összes hangmintából STFT, log-mel és MFCC spektrogramokat generál, és NumPy mátrixokba menti.
06_extract_features/	Alkönyvtár a három jellemzőtípus önálló kinyerő szkriptjeivel: 06_extract_stft.py, 06_extract_log_mel.py, 06_extract_mfcc.py.
07_realtime_recognition.py	A valós idejű hangszerfelismerő modul: mikrofonbemenet, csúszóablak, előrejelzés és megjelenítés.
10_train_model_colab/	A Google Colab felhőkörnyezetben futtatott tanító szkriptek: 10_train_stft.py, 10_train_log_mel.py, 10_train_mfcc.py.
utils/	Segédmodulok: augmentation_utils.py (az augmentációs lánc függvényei) és feature_utils.py (jellemzőkinyerési segédfüggvények).

9.3. Fejlesztői és futtatási környezet

Az alábbi táblázat tartalmazza a projekt fejlesztéséhez és futtatásához szükséges szoftverfüggőségeket.

9.3.1. táblázat: A fejlesztői környezet főbb összetevői

Szoftver / Könyvtár	Verzió	Felhasználás
Python	3.10+	A teljes feldolgozási csővezeték nyelve
TensorFlow / Keras	2.x	A 2D CNN modell építése, tanítása és mentése
Librosa	0.10+	Hangfájlok betöltése, STFT, mel-spektrogram és MFCC kinyerés
NumPy	1.24+	Jellemzőmátrixok tárolása és manipulálása
SoundFile / SoundDevice	–	Hangfájlok olvasása/írása és valós idejű mikrofon-hozzáférés
Matplotlib	3.x	Diagramok, spektrogramok és kiértékelési ábrák készítése
Google Colab	–	GPU-gyorsított modelltanítás felhőkörnyezetben
Git	2.x	Verziókezelés és a projekt előrehaladásának dokumentálása
L ^A T _E X (TeX Live 2026)	–	A szakdolgozat szedése a Sapientia sablon alapján